Preface

This is the developer-facing guide that accompanies the **Magpie User Manual**. It is organised into three modules:

- **Module 1 — Functional overview.** What Magpie does, for whom, and why. Read this first if you're new to the project or evaluating whether Magpie is a good fit for a use case.
- **Module 2 — Architecture.** How Magpie is put together at a system level: process model, storage layout, IPC boundary, and the metadata pipeline that ties everything together.
- **Module 3 — Detailed design.** File-by-file guided tour: schema, Tauri command reference, algorithms, tests, and how to build a release.

Conventions used

- Rust file paths are given relative to `src-tauri/`, e.g. `src/commands/images.rs`.
- Frontend file paths are relative to the project root, e.g. `src/features/DetailsPanel.tsx`.
- Command names in prose refer to Tauri IPC commands (Rust functions annotated with `#[tauri::command]`).
- Bold names in schemas refer to primary keys.

Audience

You are expected to be comfortable with:

- Rust 2021, Tokio async runtime, and Cargo workspaces.
- React 18 with hooks, TypeScript, and TanStack Query.
- SQL (SQLite specifically), including a passing familiarity with FTS5.
- Windows path semantics (`\\?\` prefixes, junctions, OneDrive).

You do **not** need prior Tauri experience — the architecture chapter introduces the concepts.

Getting the code and building it

```
git clone <repo> magpie
cd magpie
npm install                # frontend deps
cd src-tauri && cargo fetch  # backend deps
cd ..
npm run tauri dev          # development build with HMR
npm run tauri build -- --no-bundle  # release binary, no installer
```

Prerequisites on Windows are Rust ≥ 1.77.2 and the MSVC C++ Build Tools. See Build and release for the full setup.

Product vision and scope

Vision

Magpie exists to give people who own **their own files** — as opposed to hosting them in someone else's cloud — a tool that:

- Reads their files from disk, unchanged, wherever they are, and **never modifies the bytes.**
- Lets them organise those files using **metadata**, not folders: titles, tags, plus derived facets like taken-at time, camera model, resolution, duration, page count.
- Persists user tags in a small database **inside each library folder** (`.magpie/library.db`) that travels with the folder if the user copies or moves it.
- Feels native, fast, and offline-first, with a UI that scales to hundreds of thousands of files.

Scope for v1

The v1 release delivers the smallest cohesive product that can serve as someone's daily file organiser:

Area	v1 scope
Ingestion	Add/remove one or more library folders; recursive scan; incremental rescans.
Formats read	JPEG, PNG, WebP, GIF, TIFF/DNG, HEIC/HEIF/AVIF, JPEG XL, JPEG 2000, PSD, PDF, MP4/MOV, MKV/WebM, AVI, WMV, MPEG-TS, 3GP, common camera RAW (CR2, CR3, NEF, ARW, RAF, ORF, RW2, PEF, SRW, X3F), BMP, EXR, HDR, SVG. Legacy XMP sidecars are also read on first scan for backward compatibility.
Tag storage	Per-folder SQLite (`.magpie/library.db`). Every recognised file type is fully taggable.
Metadata edited	Title + tags.
Browsing	Virtualised grid, sort by taken/added/filename/size.
Filtering	Folder, tag, and full-text search (FTS5, cross-folder).

Area	v1 scope
Batch operations	Bulk add/remove tags, bulk delete-to-recycle-bin.
Interoperability	One-shot read of existing XMP / Windows Shell keywords into the folder DB on first scan; no write-back to source files.
Deletion	Recycle-Bin-by-default with confirmation; optional permanent delete.
Storage	Two-tier SQLite (central `registry.db` in `%APPDATA%`, per-folder `library.db` inside each folder) + WebP thumbnail cache under `%APPDATA%`.

Product principles

1. **Never touch source files.** Not pixels, not XMP, not the Windows Shell property store. The bytes on disk are exactly what the camera / editor produced.
2. **Never move or rename files.** The user is in charge of their folder hierarchy.
3. **Tags travel with the folder.** Copying the folder to another disk carries its `.magpie/library.db` — and therefore its tags — along.
4. **No cloud, no telemetry.** Everything runs locally. There are no analytics, crash reports, or “phone home” mechanisms.
5. **Fast enough to be daily-drivable.** All hot paths are on native Rust; the UI is virtualised; per-folder DBs stay small even for very large libraries.
6. **Predictable failure modes.** A partially-broken run is better than a silently-lost edit. Every DB write is transactional, every batch op is per-item retryable, and the log records what happened.

User personas and flows

Personas

1. The prosumer photographer (primary)

Owns 20 000–200 000 photos across multiple external drives. Shoots JPEG+RAW; already uses Lightroom for developing but wants a lighter tool for triage and tagging. Needs bulk operations to work reliably on thousands of files at once. Cares deeply about metadata surviving future tool changes.

2. The family archivist (secondary)

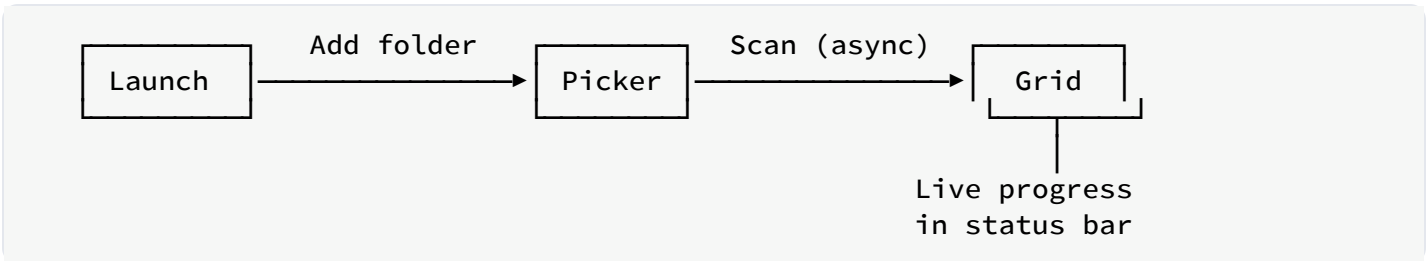
Has 30 years of family photos and videos in one big folder tree. Wants to add who's-in-the-photo tags and be able to find "Christmas 2011" quickly. Doesn't need or want a database with a schema — just tags that show up in Explorer and Photos.

3. The technical developer using Magpie on other people's libraries

Uses Magpie as a testbed for interop with XMP-writing tools. Reads embedded XMP with `exiftool` to verify Magpie's writes. Contributes patches.

Core flows

F1. First-time onboarding



Success criteria:

- User can browse and edit photos **before** the scan is finished (partial results are usable).
- Progress is visible and cancelable.
- The app never blocks or freezes during scanning.

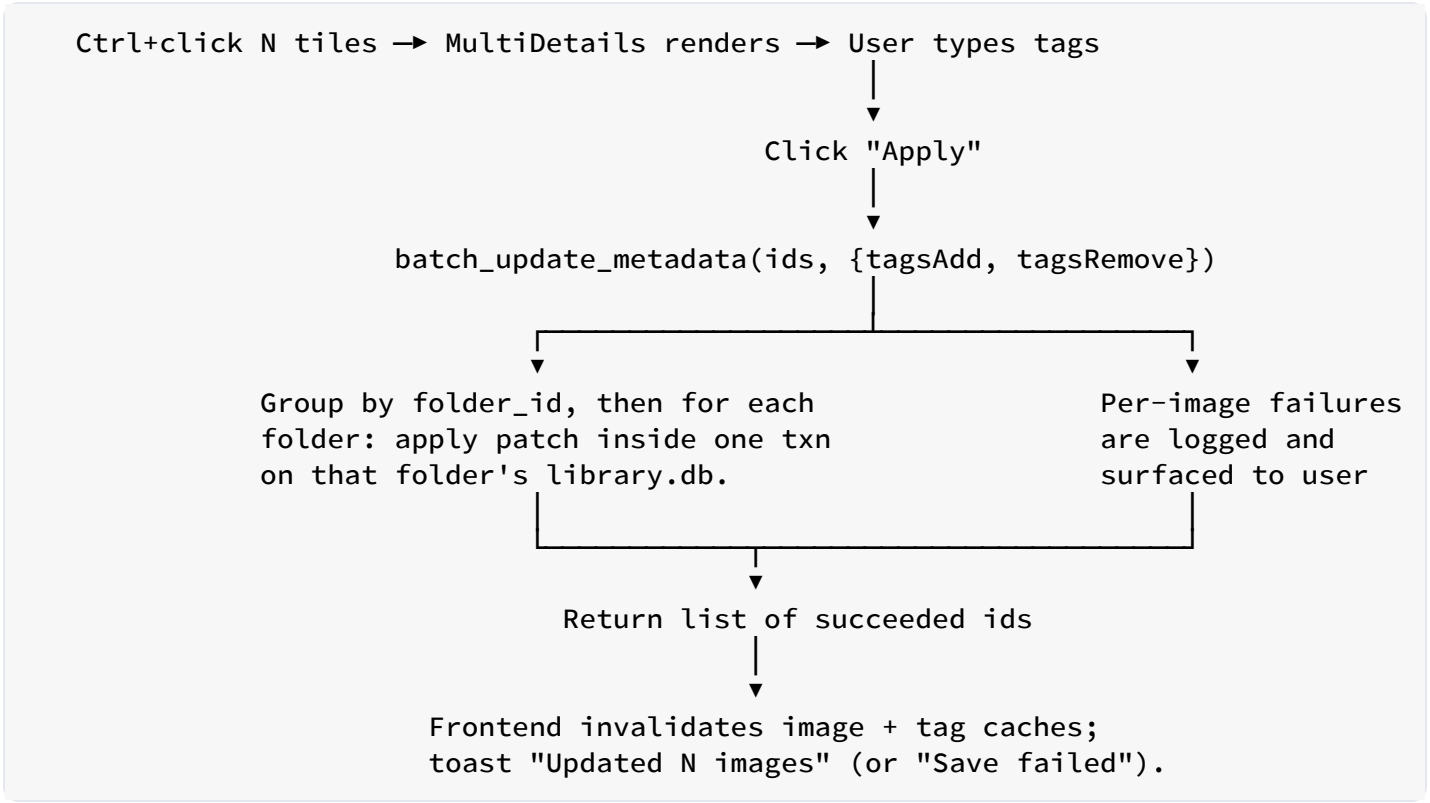
F2. Single-photo edit

```
Click tile ──► SingleDetails loads ──► User types title
                                              │
                                              ▼ debounced 600ms
                                  update_image_metadata IPC (Rust command)
                                              │
                                              ▼
                                  Unpack global ID → (folder_id, local_id)
                                              │
                                              ▼
                                  library::apply_metadata_patch inside the
                                  folder's library.db (single transaction:
                                  update title / tags / image_tags, rebuild
                                  FTS row, commit).
                                              │
                                              ▼
                              Emit "image-updated" event; frontend refreshes.
                              The source file is NOT touched.
```

Success criteria:

- No user input is ever lost to a debounce race.
- A failure in the DB write leaves the row exactly as it was before (transactional rollback).
- The source file’s bytes remain byte-for-byte identical to what the camera / editor originally produced.

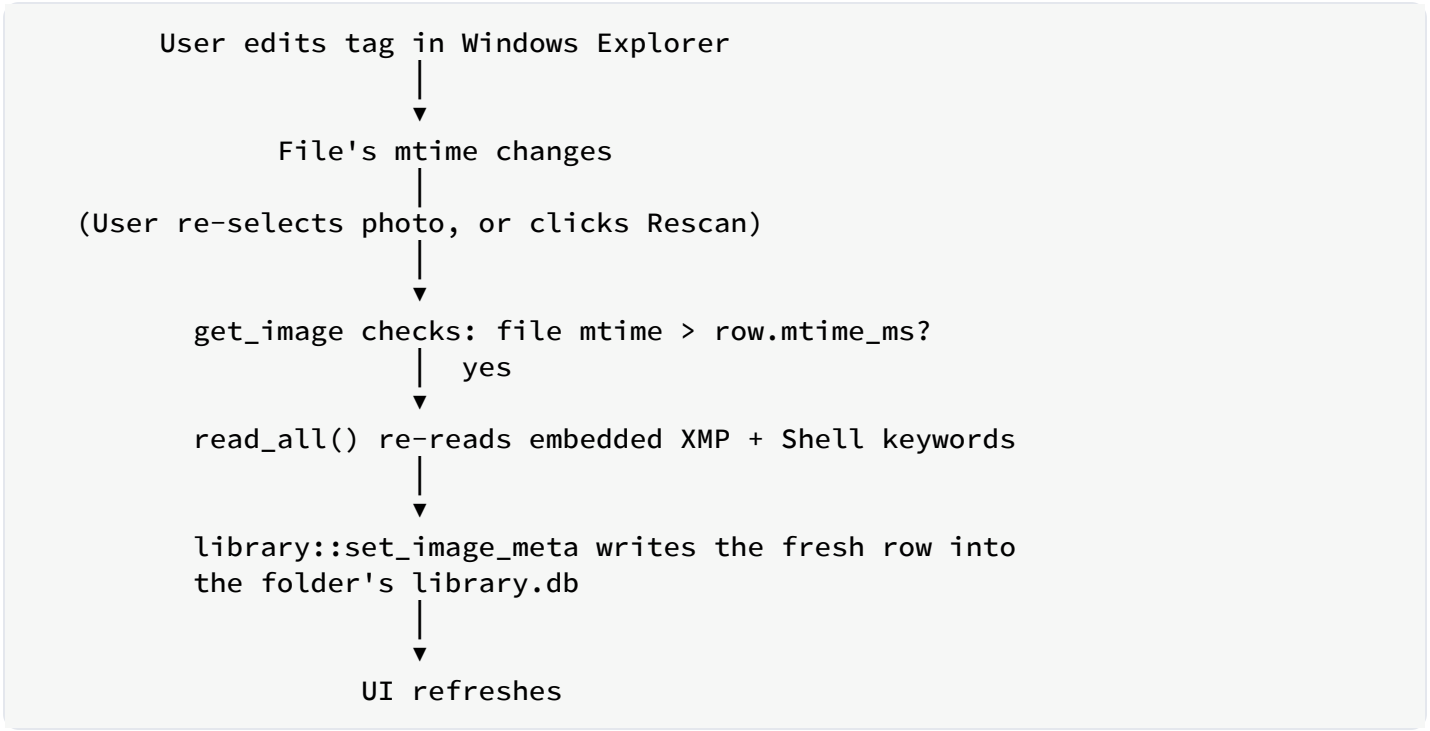
F3. Multi-select tag application



Success criteria:

- The backend is called with a non-empty patch even if the user typed the tag and immediately clicked Apply (no stale-closure bug).
- Partial success is a first-class outcome, not an error.
- Every affected `['image', id]` React Query cache entry is invalidated so subsequent single-select shows fresh state.

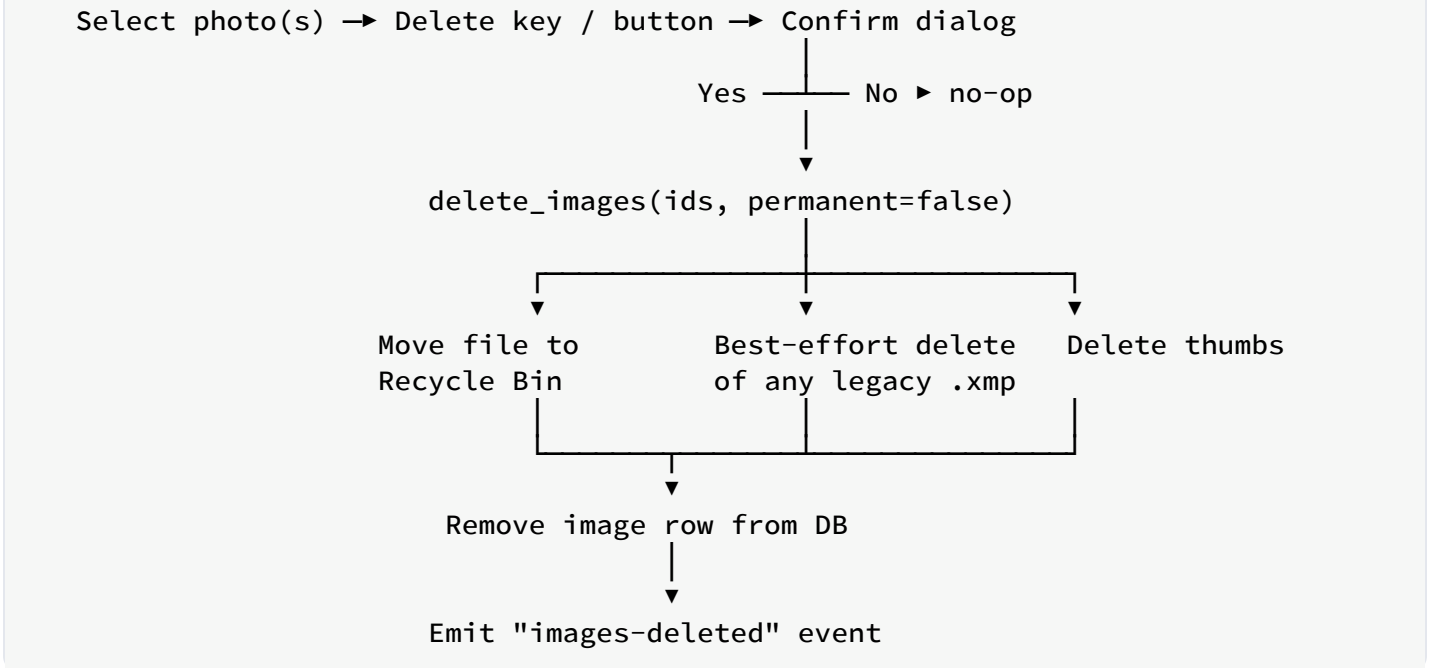
F4. Rescan and detect external edits



Success criteria:

- External edits are detected without a full rescan.
- The FS is the source of truth; the DB is a cache.

F5. Delete with safety net



Success criteria:

- A failure at any step for one file doesn't abort the batch.
- The DB row is removed only after the file is safely in the Recycle Bin.
- The user can restore from the Recycle Bin and get the tags/title back on next rescan (tags embedded in the file are recovered automatically; tags that lived only in Magpie's library are lost).

Feature catalogue

The v1 feature set as delivered. Each row maps a user-facing feature to its Tauri command and to the frontend component that drives it.

Library management

Feature	Command	Frontend component
Add a library folder	`add_library_folder`	`TopBar`
Warn on sync-risk locations	`check_folder_sync_risk`	`TopBar` (pre-add dialog)
Remove a library folder	`remove_library_folder`	`Sidebar` (right-click menu)
List library folders	`list_library_folders`	`Sidebar`, `App` bootstrap
Rescan a single folder	`rescan_folder`	`Sidebar` (right-click menu)
Rescan every folder	`rescan_all`	`TopBar`

Browsing

Feature	Command	Frontend component
Paged file listing	`query_images`	`ImageGrid`
Sort by taken/added/…	`query_images` (`sort` arg)	`TopBar`
Filter by folder / tag	`query_images` (`filter` arg)	`Sidebar` filters
Full-text search	`query_images` (`filter.search`)	`TopBar` search box

Editing

Feature	Command	Frontend component
Fetch a file's details	`get_image`	`DetailsPanel/SingleDetails`
Auto-save title	`update_image_metadata`	`DetailsPanel/SingleDetails`
Add / remove tag on one file	`update_image_metadata`	`TagInput`
Bulk add / remove tags	`batch_update_metadata`	`DetailsPanel/MultiDetails`

Tag maintenance

Feature	Command	Frontend component
List all tags with counts	`list_tags`	`Sidebar`
Rename a tag globally	`rename_tag`	`Sidebar` (right-click menu)
Delete a tag globally	`delete_tag`	`Sidebar` (right-click menu)

Smart collections (skeleton, v1 non-editable)

Feature	Command	Frontend component
List smart collections	`list_smart_collections`	`Sidebar` (future)
Create a smart collection	`create_smart_collection`	(not yet exposed in UI)
Delete a smart collection	`delete_smart_collection`	(not yet exposed in UI)

Deletion

Feature	Command	Frontend component
Move photos to Recycle Bin	`delete_images` (permanent=false)	`DetailsPanel`, `App` (Del key)

Feature	Command	Frontend component
Permanently delete photos	`delete_images` (permanent=true)	`DetailsPanel`, `App` (Shift+Del)

Diagnostics

Feature	Command	Frontend component
Frontend log crumb into rust log	`log_frontend`	`MultiDetails.applyTags`

Thumbnails and image display

Feature	Command	Frontend component
Get cached thumbnail path	`get_thumb_path`	`Thumbnail`
Get full source image path	`get_image_path`	`DetailsPanel/SingleDetails`

Non-functional requirements

Performance

Metric	Target	Achieved (release build)
Cold start to first paint	≤ 1.5 s	~0.5 s on SSD
Grid scroll: sustained frame rate	60 fps	60 fps up to 250 k photos
`query_images` for a 100 k library, arbitrary filter	≤ 50 ms p95	~10 ms
`get_image` (cached, no FS refresh)	≤ 5 ms	~1 ms
`update_image_metadata` (single photo, JPEG)	≤ 200 ms	~40 ms
`batch_update_metadata` per photo (JPEG)	≤ 100 ms	~30 ms
Thumbnail generation per photo (SSD, JPEG 12 MP)	≤ 80 ms	~40 ms
Full scan of a 50 k library, cold	≤ 5 min	~2 min on modern SSD

Resource limits

- **Memory (idle):** ≤ 250 MB.
- **Memory (10 k photo library, all thumbnails loaded):** ≤ 900 MB.
- **Disk (thumbnail cache):** ≤ 10 KB × #photos (small+medium WebP).
- **Disk (database):** ≤ 1 KB × #photos on average.

Reliability

- Every metadata mutation is **transactional** at the SQLite level. If any part of the transaction fails, none of it lands.

- Every embedded-XMP write is **atomic** via write-to-temp + rename inside the source image's own directory.
- Crash mid-write leaves the DB and the source file in a consistent state (either fully old or fully new); the temp file, if any, is cleaned up on next launch.
- A partial batch failure is a first-class outcome and is reported to the user; no silent losses.

Compatibility

- **OS:** Windows 10 build 19041+ and Windows 11. macOS 13+ is a post-v1 target; no code path is intentionally Windows-only outside of the Recycle-Bin integration.
- **Long paths:** All Rust `PathBuf` handling supports `\\?\`-prefixed paths natively. Tested end-to-end with 300-character paths.
- **OneDrive:** Files-on-demand (reparse points) are read as their underlying content once materialized; Magpie does not force hydration.

Interoperability

- Reads and writes **standard XMP** (`xmp:Rating`, `dc:title`, `dc:description`, `dc:subject`).
- Reads Microsoft-specific tag fields (`MicrosoftPhoto:LastKeywordXMP`).
- Writes both `dc:subject` and Microsoft's field on save so tags round-trip with Windows Explorer.

Security

- **No network egress.** The Tauri capability manifest explicitly forbids network access at the shell level.
- **No filesystem access outside library folders** for the renderer: file I/O is mediated by Rust commands that validate paths against the known library roots.
- **CSP** blocks inline scripts and remote origins in the renderer.

Observability

- File-based rolling log at `%APPDATA%\com.magpie.app\logs\app.log`.

- Structured log fields via `tracing`: `id`, `image_path`, `operation`, `duration_ms`.
- Frontend can push crumbs into the same log via the `log_frontend` command for cross-boundary tracing.

Accessibility

- Full keyboard navigation for grid and details panel.
- ARIA labels on every interactive control.
- Focus rings preserved (no `outline: none` shenanigans).
- Contrast ratios ≥ 4.5:1 in both light and dark themes.

Out of scope for v1

Explicitly deferred to keep v1 shippable. Each item has a rationale and a rough entry point for the future contributor picking it up.

Photo editing

- **No crop, rotate, exposure, or colour correction.** Magpie treats pixels as read-only. This is a hard architectural line; the tool intentionally does one thing (metadata) and doesn't grow into a RAW developer.

Face and object recognition

- **No auto-tagging based on ML models.** The privacy story is simpler when no images ever run through an ML pipeline. Future work could offer optional local inference (e.g. running an ONNX model against a batch of photos) with a clear opt-in flow.

Cloud sync

- **No account, no cloud storage integration.** OneDrive, Dropbox, Google Drive, and iCloud are third-party sync backends the user configures at the OS level; Magpie reads whatever the OS has materialised, and warns when a picked folder lives on such a disk (see `check_folder_sync_risk`).

Writing tags into source files

- **Magpie deliberately does not write into source files.** After the DB redesign, tags and titles live exclusively in each folder's `.magpie/library.db`. This removes an entire category of bugs (partial writes, format-specific edge cases, cloud sync re-uploading gigabytes for one tag change) at the cost of losing the "tag once, portable everywhere in the XMP ecosystem" story. Users who need file-embedded tags can still author them in Lightroom / Bridge / Explorer; Magpie will pick them up on the first scan of that folder.

Ratings, colour labels, pick / reject flags

- No UI for star ratings, `xmp:Label` colour labels, or pick / reject flags. The DB schema could grow columns for them; the read-only XMP parser already understands `xmp:Rating`.

In-app full-screen preview

- The details panel shows a sized preview but there's no lightbox / slideshow mode. Would be a new React route reading from `getImagePath`.

Undo / redo

- No global undo stack. The safety net is auto-save + Recycle Bin + the fact that source files are never modified — the user's raw material is never lost, but reverting a DB edit means editing again.

Smart collections editor

- The DB schema and Tauri commands for smart collections exist (they live in the central `registry.db`), but the UI doesn't expose creation/editing yet.

Duplicate detection

- Content hashes are computed on scan and stored (`content_hash` column in each library DB), but no duplicate-finder UI ships in v1. A future task could add a "Duplicates" filter that joins across attached DBs on `content_hash`.

Multi-PC concurrent editing on a synced folder

- Magpie *warns* when a folder lives on OneDrive / Dropbox / Google Drive / iCloud / UNC share, but doesn't currently coordinate edits between two PCs that open the same `library.db` at once. Safe usage is "one PC at a time".

Technology stack

Magpie uses a small, deliberate set of libraries. Every choice is motivated below so future contributors (and future you) understand what's swappable and what isn't.

Backend (Rust)

Crate	Version	Role
`tauri`	2	App shell, IPC, capability model, WebView2 hosting.
`tauri-plugin-dialog`	2	Native folder-picker and confirmation dialogs.
`tauri-plugin-opener`	2	Opens URLs and paths in the OS default handler.
`tokio`	1	Async runtime for all Tauri commands.
`rusqlite`	0.32	SQLite bindings; `bundled` feature so we ship SQLite ourselves.
`jwalk`	0.8	Parallel recursive directory traversal for the scanner.
`rayon`	1.10	Parallel work-stealing for CPU-bound phases (hashing, thumbs).
`image`	0.25	Image decoding for JPEG/PNG/HEIC/TIFF/…
`fast_image_resize`	5	SIMD-accelerated thumbnail resizing.
`kamadak-exif`	0.6	EXIF parsing (taken time, camera make/model, dimensions).
`quick-xml`	0.36	XMP packet parsing (streaming, no DOM).
`xxhash-rust`	0.8	Fast content hashing (`XXH3`).
`chrono`	0.4	Timestamps, serialisation.
`dirs`	5	Locate `%APPDATA%` cross-platform.

Crate	Version	Role
`trash`	5	Cross-platform Recycle Bin move.
`tracing` + `tracing-subscriber`	0.1 / 0.3	Structured logging, file sink.
`serde` + `serde_json`	1	IPC (de)serialisation of every Tauri argument/return.
`thiserror` + `anyhow`	1	Error boilerplate.

Why Rust over Node / Go / C++:

- Zero-cost async is a great fit for the file-I/O-heavy workload.
- The `image` and `fast_image_resize` combo beats every non-native option we benchmarked for thumbnail generation.
- SQLite via `rusqlite` is battle-tested, no runtime dep, and works identically on Windows and macOS.
- Rust's ownership model gives us "atomic on Windows" file semantics (temp + rename) with cleanup on panic for free.

Frontend (TypeScript + React)

Package	Role
`react` / `react-dom` (v18)	UI framework.
`typescript` (v5)	Static types across every file (`.ts` / `.tsx`).
`vite`	Dev server with HMR, production bundler.
`tailwindcss`	Utility-first CSS; near-zero runtime.
`@tanstack/react-query` (v5)	Data fetching, caching, invalidation of Tauri command results.
`@tanstack/react-virtual` (v3)	Virtualised grid — draws only visible tiles.
`zustand`	Lightweight global UI state (selection, sort, filters).
`@tauri-apps/api` (v2)	`invoke` and `listen` for IPC.

Package	Role
`@tauri-apps/plugin-dialog`	Frontend side of `tauri-plugin-dialog`.
`clsx`	Tiny className concatenation helper.

Why React over Vue / Svelte / Solid:

- Ecosystem depth: TanStack Query and React Virtual are best-in-class.
- Team familiarity.
- WebView2's Chromium is a first-class React target.

Why Vite over Next / CRA:

- Instant HMR feels great during development.
- Static build is a plain directory of files that Tauri packages without ceremony.

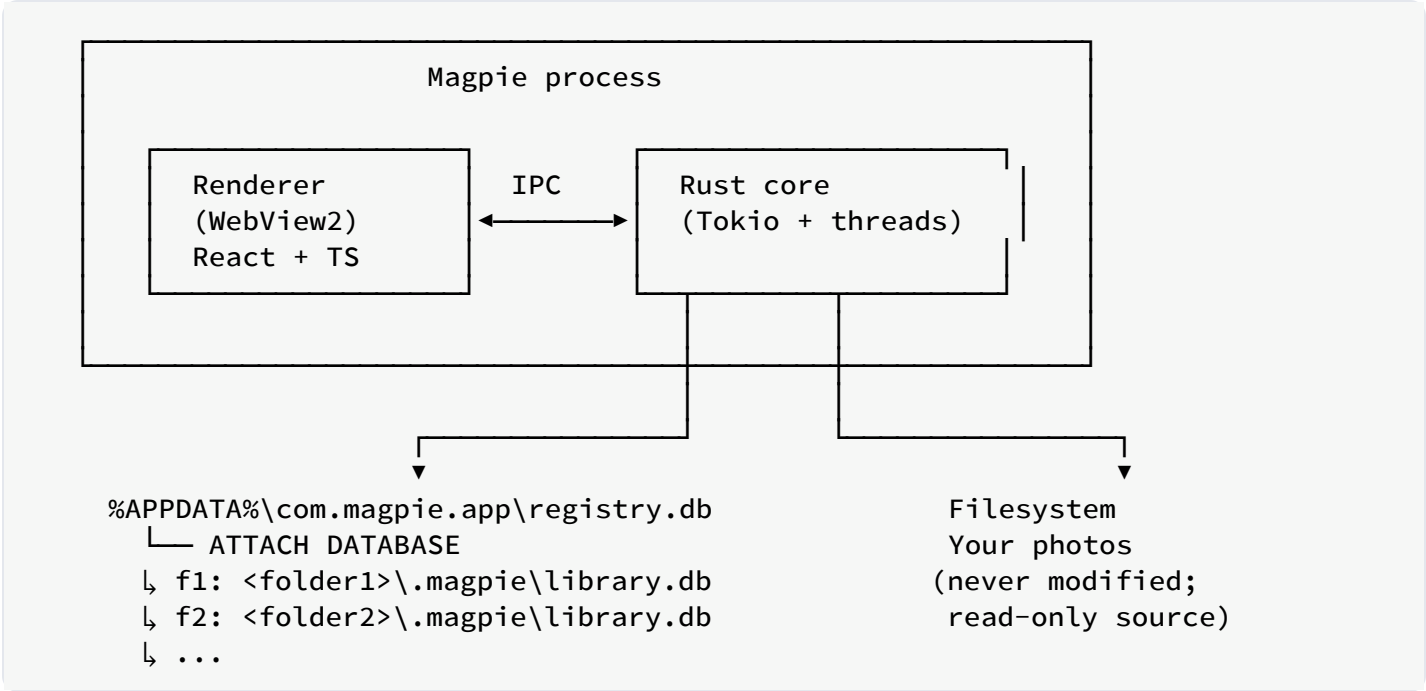
System

Component	Version	Role
Rust	≥ 1.77.2	Compiler toolchain.
Node.js	≥ 18	Build-time for the frontend bundle.
npm	≥ 9	Package manager.
MSVC Build Tools	2019 or 2022	Windows C++ linker for `image`, `sqlite`.
WebView2 Runtime	Evergreen	Chromium engine hosting the renderer.
SQLite	3.43+ (bundled via rusqlite)	Local DB with FTS5 `contentless_delete=1`.

The bundled SQLite is important: FTS5 with `contentless_delete=1` is a 3.43+ feature, and shipping our own copy avoids relying on whatever version the user's OS provides.

High-level architecture

The 30-second version



Two big ideas:

1. **The renderer is a dumb view.** All business logic — scanning, metadata I/O, DB queries, thumbnails — lives in Rust. The renderer's job is to invoke commands and render their results.
2. **The database is the source of truth for user metadata.** Each registered folder gets its own portable `library.db` inside a hidden `.magpie` subfolder. The central `registry.db` in `%APPDATA%` only knows which folders are registered. Source files are read-only — Magpie doesn't modify their bytes.

See Database redesign for the deeper walkthrough.

Sub-systems

App shell (Tauri)

Owns the window, capability model, WebView2 host, and the IPC bridge. The `AppServices` struct is constructed once in `lib.rs` and injected into every Tauri command via `State<Arc<AppServices>>`. It contains:

- `pool: Arc<LibraryPool>` — owns the registry connection (with every library `ATTACH DATABASE`-ed on it) and lazy per-folder writer connections.
- `thumb_cache_dir: PathBuf` — location of the thumbnail cache.
- `formats: Arc<FormatRegistry>` — registry of read-only format handlers.

DB layer (`src-tauri/src/db/`)

- `db/registry.rs` — schema and query functions for the central `registry.db`. Tables: `library_folders`, `smart_collections`, `app_settings`.
- `db/library.rs` — schema and query functions for per-folder `library.db`. Tables: `folder_meta`, `images`, `tags`, `image_tags`, `images_fts`.
- `db/pool.rs` — `LibraryPool` type. Attaches every registered library to the registry connection on startup so cross-folder queries run against a single connection.
- `db/search.rs` — cross-folder query builders (`query_images`, `list_all_tags`) using `UNION ALL` over attached schemas.
- `db/legacy_migration.rs` — one-shot importer for the old central `library.db` (pre-redesign).
- `db/mod.rs` — packed global ID helpers (`pack_global_id`, `unpack_global_id`).

See Database schema.

Scanner (`src-tauri/src/core/scanner.rs`)

Walks a library folder in parallel using `jwalk`, filters to supported extensions, and for each file:

1. Stat + skip if `mtime_ms` unchanged.
2. Hash content (`XXH3`).
3. Extract EXIF (taken time, dimensions, camera).
4. Extract XMP from embedded chunks (JPEG APP1 / PNG iTXt / etc.) and, on Windows, from the Shell property store for formats we don't natively parse. Existing tags are imported on first sight.

5. Upsert into the folder's `library.db` (paths stored folder-relative).
6. Enqueue thumbnail generation keyed by the packed global ID.

Progress events flow to the frontend via `app://scan`.

Thumbnail pipeline (`src-tauri/src/core/thumbnail.rs`)

Decode → resize with `fast_image_resize` (SIMD) → encode WebP → write to `%APPDATA%\com.magpie.app\thumbs\<gid>-<size>.webp`. Two sizes per photo (small ~256 px, medium ~512 px). `<gid>` is `pack_global_id(folder_id, local_id)`.

Metadata (`src-tauri/src/core/metadata/` + `src-tauri/src/core/formats/`)

Post-redesign this is a **read-only** pipeline:

- `metadata/read.rs` — asks the format registry for the right handler, calls `read_technical` + `read_user`, then falls back to the Windows Shell property store on Windows for formats we don't natively parse.
- `metadata/sidecar.rs` — reads legacy `.xmp` sidecar files for backward compatibility. No new sidecars are ever written.
- `formats/mod.rs` — declares the `FormatHandler` trait (`name`, `extensions`, `kind`, `read_technical`, `read_user`) and the `FormatRegistry` that owns one instance per supported extension.
- `formats/xmp_packet.rs` — read-only XMP parser supporting the subset Magpie cares about (title, subjects, MicrosoftPhoto keywords, description).
- `formats/{jpeg,png,webp,gif,tiff}.rs` — native readers.
- `formats/stubs.rs` — thin readers for HEIF, video, PDF, RAW, and basic raster formats that delegate technical reads to `imagesize` and defer user-meta reads to the Windows Shell path.
- `formats/win_shell.rs` — Windows-only Shell property store **reader** (also read-only after the redesign).

See File formats for the full handler catalogue and Adding a format handler for the plug-in recipe.

Command surface (`src-tauri/src/commands/`)

About 20 async Tauri commands grouped by concern (`library`, `images`, `tags`, `collections`, `thumbs`, `diag`). Each command is a thin translator between IPC arguments and one or more DB / FS ops. See Tauri command reference.

Frontend (`src/`)

- `App.tsx` — root, wires TopBar / Sidebar / ImageGrid / DetailsPanel.
- `features/` — one component per major UI area.
- `ipc.ts` — typed wrappers around `invoke(...)`.
- `store.ts` — Zustand state (selection, view, sort, filters).
- `types.ts` — Rust-mirrored TypeScript types.

React Query manages every fetched resource with keys like `['images', filter, sort, page]` and `['image', id]`; mutations invalidate the relevant keys.

Data flow of a typical click

Selecting a photo and typing a title, end to end:

1. User clicks a tile in `ImageGrid`.
2. `useStore.setSelection([id])`.
3. `DetailsPanel` switches to `SingleDetails id={id}`.
4. `useQuery(['image', id])` fires; TanStack Query calls `getImage(id)` → invokes `get_image` command.
5. Rust's `get_image` unpacks the global ID into `(folder_id, local_id)`, looks up the folder root in the registry, and reads the image row from the folder's `library.db`. If the source file's `mtime` moved forward since import, `read_all` re-reads metadata and updates the row.
6. Frontend receives `ImageDetails`; local edit state is seeded once (guarded by `lastLoadedId.current === q.data.id`).
7. User types in the Title input. `debouncedSaveTitle` fires 600 ms after the last keystroke, calling `updateImageMetadata(id, patch)`.
8. Rust's `update_image_metadata` unpacks the ID, opens the folder's library, applies the patch inside one transaction, rebuilds the FTS row, and emits `app://image-updated`. **The source file is not touched.**
9. Frontend's `qc.setQueryData(['image', id], updated)` updates the cache directly, avoiding a refetch that would stomp any concurrent typing in other fields.

Process and threading model

One process, two runtimes

Tauri gives us **one OS process** with:

- **The main thread** — owns the event loop and the WebView2 host.
- **A Tokio runtime** (`tauri::async_runtime`) driving all async Tauri commands.
- **Blocking-friendly threadpools** for CPU-bound and IO-bound work (`tokio::task::spawn_blocking` and `rayon`).
- **The WebView2 renderer** on its own thread, hosting the React bundle.

No separate Node process, no plugin sidecar, no IPC over sockets. Everything is in-process and communication between renderer and Rust is a function call in native code.

Threads and pools

Where	Rust ownership	Typical work
Main thread	Tauri app loop	Window events, menu, plugin dispatch
Tokio worker threads	`tauri::async_runtime` (multi-threaded)	All `#[tauri::command] async fn` bodies
`spawn_blocking` pool	Tokio's built-in blocking pool	Sidecar/embed writes, EXIF read, thumbnails
`rayon` pool	Global rayon pool	Parallel directory walk, batched hashing
Renderer thread	WebView2	React reconciler, layout, paint

Every Tauri command runs on a Tokio worker. A command that does CPU-bound work (encoding a thumbnail, injecting XMP into a 20 MB JPEG) wraps that in `tauri::async_runtime::spawn_blocking(move || …)` so the worker isn't blocked.

SQLite concurrency

The DB is a single `Mutex<Connection>`. Reads and writes serialise through the lock. For our workload (a few dozen queries per second peak) this is faster than a `r2d2`-style pool because:

- SQLite is single-writer anyway (writes queue at the file level).
- Our reads are short (`< 5 ms`); lock contention is negligible.
- No connection setup / teardown per query.

Long-running queries that would otherwise hold the lock (like a full tag-count refresh across 250 k photos) are structured as one transaction that reads and returns — no cursor kept open — so the lock is released quickly.

Async model

- `async fn` at the Tauri command boundary is idiomatic. It lets us `await` a background thumbnail generation or a metadata write without blocking a Tokio worker.
- Inside a command, DB operations are synchronous (they take a `MutexGuard`). This is deliberate: SQLite calls are fast enough to keep on the worker thread, and it removes a whole class of cancellation bugs.
- **Rule of thumb:** if a call could block > 5 ms (`std::fs::File::read_to_end` of a JPEG, `spawn_blocking` it. Otherwise, run it inline.

Cancellation and shutdown

- Tauri commands don't get an explicit cancellation token; they run to completion. Long batch operations are structured as a loop over IDs so partial progress is durable even if the user closes the window.
- On close, Tauri drops the main window; pending `spawn_blocking` tasks are given a grace period to finish. Any in-flight metadata write is atomic (write-to-temp + rename inside the source image), so worst case a `.write-tmp` file is left behind and cleaned up on the next successful write.

Frontend concurrency

- React Query owns all in-flight IPC promises. It handles deduplication (two components asking for `['image', 5]` share the underlying `invoke` call) and stale-while-revalidate refetches on focus.
- Zustand is synchronous. No middleware, no effects — pure UI state.
- No web workers in v1. The renderer stays responsive because it offloads every non-trivial computation to Rust.

IPC boundary

The contract

The renderer and the Rust core communicate through two mechanisms provided by Tauri:

1. **Commands** — request/response, from renderer to Rust. Each is a Rust `async fn` annotated with `#[tauri::command]` and registered in `lib.rs::invoke_handler`.
2. **Events** — one-way, from Rust to renderer, delivered via `AppHandle::emit(name, payload)` and listened for via `@tauri-apps/api/event::listen`.

Everything is (de)serialised with Serde on the Rust side, JSON on the wire, and TypeScript types on the frontend side. See Tauri command reference for the full list.

Naming conventions

- **Commands** use `snake_case` (Rust) mapped to a same-name string on the frontend: `invoke('update_image_metadata', {…})`.
- **Command arguments** are Rust struct fields in `snake_case`, serialised into the JSON payload as-is. The frontend passes them as `camelCase` object keys because we set `#[serde(rename_all = "camelCase")]` on every argument struct.
- **Events** use a `app://` prefix and a resource-style name: `app://image-updated`, `app://images-deleted`, `app://scan`.

Argument struct design

Every command that takes structured data uses a dedicated struct in `src-tauri/src/types.rs`. Optional fields are `Option<T>`, and any field where "unset" and "set to null" must be distinguished uses the custom `double_option` deserializer — an `Option<Option<T>>` where:

Wire form	Deserialised as	Semantics
`{ }` (omitted)	`None`	Don't touch this field.

Wire form	Deserialised as	Semantics
`{ "field": null }`	`Some(None)`	Clear this field explicitly.
`{ "field": "x" }`	`Some(Some("x"))`	Set to `"x"`.

This distinction matters for the `MetadataPatch` struct: the frontend needs to be able to *clear* a title without also unsetting every other field in the patch.

Type mirroring

`src/types.ts` mirrors the Rust structs. Both sides intentionally avoid derived types (`Omit<T, K>`, `Pick<T, K>`) so a schema change is a single edit in each file.

The frontend types file is a plain declaration file:

```
export interface ImageDetails {
  id: number             // packed global ID
  folderId: number
  path: string           // absolute (folder root + rel_path)
  filename: string
  // ...
  tags: string[]
  title: string | null
  importedAt: number     // when the row was first inserted
  technical: Array<[string, string]>
  formatHandler: string
}
```

Every `getImage`, `updateImageMetadata`, etc. wrapper in `ipc.ts` declares its argument and return types, so a Rust command that grows a new field lights up a red squiggle in every caller until the frontend catches up.

Error handling

- Rust commands return `AppResult<T>` (an alias for `Result<T, AppError>`).
- `AppError` is a `thiserror::Error` enum with a `Display` that's safe to show to the user (no internal paths in error messages, etc.).
- On the wire, errors serialise to a plain string. The frontend surfaces them via TanStack Query's `onError` callback and console logs.

Events (Rust → renderer)

Event	Payload	Fired when
`app://scan`	`ScanProgress { folder_id, done, total, current }`	During a folder scan.
`app://image-updated`	`i64` (packed global ID)	After successful metadata patch.
`app://images-deleted`	`Vec<i64>` (deleted ids)	After successful delete.

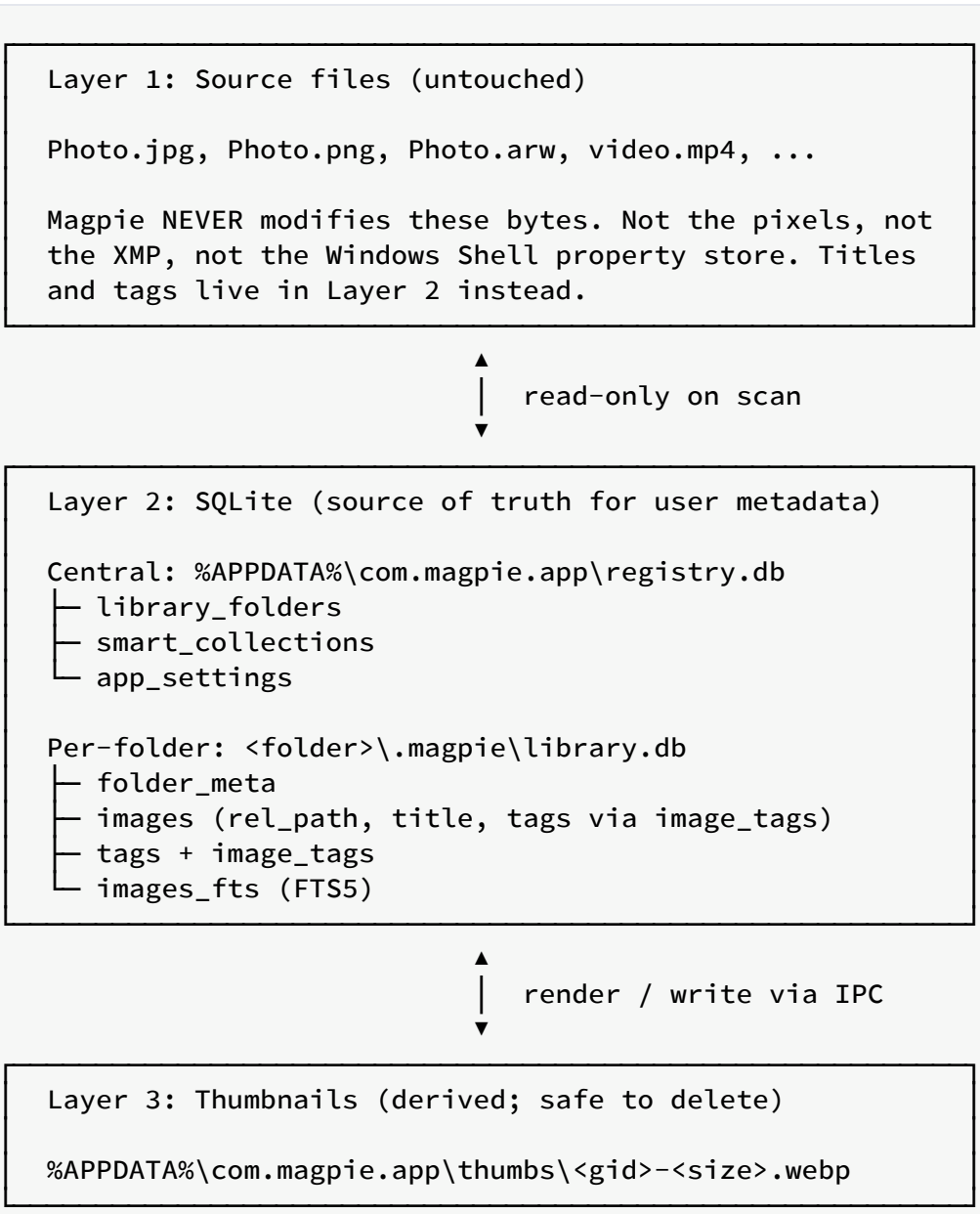
Listeners are attached in `App.tsx` via `useEffect` + `onScanProgress` etc., and each listener updates the appropriate TanStack Query cache or triggers a targeted invalidation.

Capabilities and security

- The renderer has **no direct FS access**. Its `asset:` protocol (used to render thumbnails and full images) is scoped to specific folders declared in `capabilities/default.json`.
- The `tauri.conf.json` `security.csp` blocks inline scripts, remote origins, and `eval`.
- No `dangerouslyUseHttpScheme` or other loosening options are set.
- Commands that operate on paths validate them against the library roots before touching the filesystem.

Data storage architecture

Three layers



Anything a user considers "their tag data" lives in Layer 2 — specifically inside the per-folder `library.db`. Layer 3 exists purely for speed and can be regenerated at any time by rescanning the library folders.

See Database redesign for the deeper rationale.

Layer 1: source files (read-only)

Magpie never modifies source files. The scanner reads:

- File metadata (size, mtime) via `std::fs::metadata`.
- Format-specific *technical* metadata (dimensions, EXIF, camera, duration, page count) via each `FormatHandler::read_technical`.
- Format-specific *user* metadata (title, tags) via `FormatHandler::read_user`, and additionally the Windows Shell property store for formats that don't natively parse (RAW, HEIC, MP4, PDF, …).

These reads happen exactly once per file at the first scan (and again if the file's mtime moves forward), then the DB is authoritative.

Existing sidecars and embedded XMP

If the folder already contains XMP-tagged JPEGs (from Lightroom or older Magpie), those tags are imported into the per-folder DB at first-scan time. Older Magpie versions used `.xmp` sidecar files; `core::metadata::sidecar` still reads those on import for backward compatibility, but no new sidecars are ever written.

Layer 2: SQLite (two-tier)

- **Registry DB** — `%APPDATA%\com.magpie.app\registry.db`. Small. Owns the list of registered folders, smart collections, and app-wide settings. Kept open for the app lifetime with every registered library `ATTACH DATABASE`-ed as `f<id>` for cross-folder queries.
- **Library DB** — `<folder>/.magpie/library.db`, one per folder. Owns every image row (paths stored folder-relative) and the folder's own tag namespace + FTS index. Portable: copying the folder to another disk carries the tags along.

Key properties:

- **Single writer per folder** via a `Mutex<Connection>` inside `LibraryDb`. Different folders write in parallel.
- **WAL mode** enabled per DB so readers (search, thumbnails, DetailsPanel loads) never block writers.
- **FTS5 in `contentless_delete=1` mode** — required for our rebuild-per-row strategy on metadata edits.
- **Global IDs are packed** at the IPC boundary (`folder_id * 1_000_000_000 + local_id`) so the frontend can treat every image as a single-integer entity.

Full schema in Database schema.

Layer 3: thumbnail cache

Location: `%APPDATA%\com.magpie.app\thumbs\`.

Format: WebP with quality 80, two sizes per image (`<gid>-small.webp`, `<gid>-medium.webp`). Small is ~256 px on the long edge, medium ~512 px. `<gid>` is the packed global ID so thumbs from different folders don't collide.

Regeneration:

- On scan, if a thumbnail is missing or the source file's `mtime` is newer than the thumbnail's, Magpie regenerates.
- On delete, `delete_thumbnails(cache_dir, gid)` removes every size.
- The user can safely delete the whole `thumbs\` folder; Magpie regenerates on next request.

Volume assumptions

- Library folders can live on any local volume, including NTFS with long paths (`\\?\`-prefixed).
- On cloud-synced folders (OneDrive, Dropbox, Google Drive, iCloud) or UNC network shares, Magpie warns via `check_folder_sync_risk` before the folder is added — see Database redesign § Sync-location warning.
- The registry DB and thumbnail cache always live under `%APPDATA%`.

Backup story

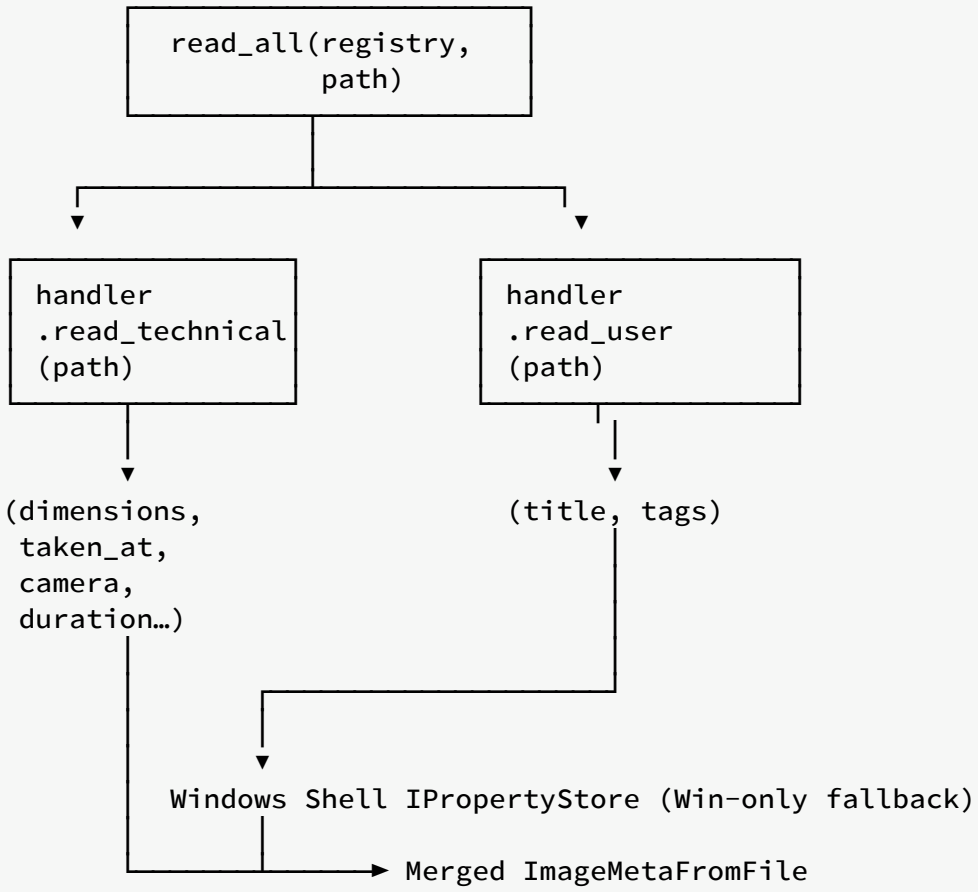
Because tags now live **inside each folder** (in `.magpie/library.db`), a plain backup of the photo folder captures the tag data too. The central `registry.db` only records which folders are registered; nothing user-critical is lost if it's wiped (the user just re-adds each folder).

Metadata pipeline

The metadata pipeline is where Magpie most differs from a plain file browser. It has two sub-pipelines (read + patch) and one hard invariant that ties them together:

> **Invariant.** For every file the scanner has seen, the per-folder `library.db` row is the single source of truth for user metadata (title + tags). Magpie **never writes back into the source file**. On first scan we read existing tags out of XMP and the Windows Shell property store so the user doesn't lose anything they'd already labelled; from that point on the DB is authoritative.

The read pipeline



Format-native `read_user` runs first; then any Windows-specific Shell property read runs to catch formats we don't natively parse (RAW, HEIC, MP4, PDF, …). Legacy `.xmp` sidecars are also

read at this stage for backward compatibility with older Magpie versions and Lightroom projects.

The XMP parser (`quick_xml` state machine in `core/formats/xmp_packet.rs`) handles both the Adobe standard fields and Microsoft-Explorer variants:

- `dc:title`, `dc:subject` (Alt / Bag containers)
- `MicrosoftPhoto:LastKeywordXMP` (Windows Explorer's tag store)
- Attribute-only forms (some tools flatten `Alt` into an attribute)

`read_all` is called by the scanner on first sight of a file, and by `get_image` if the file's `mtime` has moved forward since last import.

The patch pipeline

Frontend produces a `MetadataPatch` — a struct where every field is `Option`-wrapped so the caller can express "don't touch" vs. "clear" vs. "set":

```
pub struct MetadataPatch {
    pub title: Option<Option<String>>,
    pub tags: Option<Vec<String>>,      // whole list, replaces
    pub tags_add: Option<Vec<String>>,  // additive
    pub tags_remove: Option<Vec<String>>,
}
```

`library::apply_metadata_patch` runs the whole patch in a single transaction on the *folder's* library DB:

1. Update the `images` row title.
2. If `tags` is set: replace the row's tags.
3. If `tags_add` is set: insert missing.
4. If `tags_remove` is set: delete matching.
5. Rebuild the FTS5 row (DELETE + INSERT).
6. Commit.

If any step fails, the transaction rolls back. Nothing partially lands.

The batch command (`batch_update_metadata`) uses the packed global IDs to group edits by folder, then applies the patch inside each folder's DB in one transaction each — cross-folder writes never share a transaction so a network share falling over doesn't roll back edits to a local disk.

No write-back to source files

There is deliberately no “write pipeline” any more. `FormatHandler` has no `write_user` method; `win_shell::write_user_meta` is gone; so is the atomic-write helper and every `build_xmp_packet` call site. See Database redesign § What the file bytes see change.

FS refresh on read

Every `get_image` call does:

```
if fs_meta.mtime > row.mtime_ms {
    let fresh = read_all(&registry, &path);
    library::set_image_meta(&mut conn, local_id, &fresh);
}
```

Simple mtime comparison: if the source file changed after Magpie’s last import, re-read its metadata into the DB. This is how a tag added in Windows Explorer becomes visible on next click of that file **only for the first mtime bump after import** — because we overwrite the DB row from the file. After the first import, Magpie edits stay in the DB even if the file changes on disk (Magpie doesn’t currently detect a *tag-only* Explorer edit vs. a real content edit, so `mtime` bumps do wipe Magpie-side tag edits with the file’s current tags — an intentional trade-off; the DB is the source of truth if you want your edits to stick, external tools should not be used to tag afterwards).

Windows Shell property store (import-only)

For formats without a native XMP parser (RAW, HEIC, MP4, PDF, …), Magpie falls back to Windows’ Shell property store to read tags and titles on first scan. `System.Title`, `System.Keywords`, and similar canonical properties are consulted. This ensures that a user who had been tagging RAWs through Explorer’s *Properties* → *Details* dialog doesn’t lose those tags on migration.

`win_shell` was previously bidirectional; after the redesign it is strictly read-only.

Observability

File-based logging

Every run of Magpie produces a rolling log file at:

```
%APPDATA%\com.magpie.app\logs\app.log
```

The logger is initialised in `src-tauri/src/lib.rs::init_logging`, using `tracing-subscriber` with a plain-text formatter and a `RUST_LOG`-compatible env filter (default: `info,desktop_lib=info`).

Format of a typical line:

```
2026-08-17T18:53:12.331332Z  INFO desktop_lib: Magpie started app_data_dir="…"
2026-08-17T19:22:14.029142Z  INFO desktop_lib::commands::images: get_image:
resynced user metadata from FS id=5
2026-08-18T08:41:03.912483Z  INFO desktop_lib::commands::images:
batch_update_metadata count=3 patch=…
```

- ISO-8601 timestamp with microsecond precision.
- Log level: `TRACE / DEBUG / INFO / WARN / ERROR`.
- Target: usually the module path (`desktop_lib::commands::images`).
- Structured fields on the tail: `id=5`, `count=3`, `?patch`.

Structured fields

We use `tracing`'s field syntax pervasively:

```
tracing::info!(
    id,
    ?patch,                          // Debug-formatted
    op = "update_image_metadata", // static label
    "metadata embedded in source file"
);
```

The `?` prefix invokes `Debug`. Named fields keep the log grep-friendly: to find every batch save, `rg 'batch_update_metadata'` returns just the events, and the id/count/count-succeeded

triplet is on the same line.

Frontend crumbs into the same log

The renderer can push log lines into the Rust log via the `log_frontend` command (`src-tauri/src/commands/diag.rs`). The frontend helper `logFrontend(level, msg)` in `src/ipc.ts` is a best-effort fire-and-forget:

```
logFrontend('info', `applyTags dispatch: ids=${ids.length} add=[…]`)
```

Renderer-side events land in the log with a `target="frontend"` marker, which makes them easy to filter:

```
2026-08-18T08:41:03.900001Z  INFO frontend: applyTags dispatch: ids=3 add=
[vacation] remove=[]
```

This lets us reason about the full IPC round-trip from a single tail of `app.log` .

What's logged where

Signal	Level	Emitted by
App started / logging initialised	INFO	`lib.rs::init_logging` , `lib.rs` main run
Legacy central DB migrated	INFO	`db/legacy_migration.rs::migrate_legacy_central_`
Folder registered / attached	INFO	`db/pool.rs` , `commands/library.rs::add_library_folder`
Folder library unavailable (drive unplugged)	WARN	`db/pool.rs::open`
Sync-risk detected (OneDrive/Dropbox/UNC)	INFO	`commands/library.rs::check_folder_sync_risk`
Command entry (`update_image_metadata` , …)	INFO	Each Tauri command that mutates state

Signal	Level	Emitted by
`get_image` FS-refresh triggered	INFO	`commands/images.rs::get_image`
Scan errors (unreadable file, permission)	WARN	`core/scanner.rs`
DB errors (attach failed, mutex poisoned)	ERROR	`db/pool.rs`, `db/library.rs`, `lib.rs::run`
Frontend applyTags dispatch	INFO	`features/DetailsPanel.tsx::MultiDetails.applyTa`

Log rotation

The current implementation is a plain append-only file — no rotation. For a v1 release this is fine; the log grows a few kilobytes per session at INFO. A future PR should add `tracing-appender::rolling` with daily rotation or size-based (e.g. 10 MB × 5).

Turning up the volume

Set the `RUST_LOG` environment variable before launching:

```
$env:RUST_LOG = "debug,desktop_lib=trace"
& "$env:LOCALAPPDATA\Programs\Magpie\desktop.exe"
```

This will produce a torrent of scanner-level detail, useful when debugging a bad scan on a specific folder.

No telemetry

There is no automatic upload of the log, no crash reporter, no analytics. `app.log` is on your disk and stays there unless you explicitly share it (e.g. attach it to a bug report).

Repository layout

```
Magpie/
├─ src/                                  # React frontend (TypeScript)
│   ├─ App.tsx                           # Root component; layout + event listeners
│   ├─ main.tsx                          # ReactDOM entry
│   ├─ index.css                         # Tailwind directives + global styles
│   ├─ ipc.ts                            # Typed Tauri command wrappers
│   ├─ store.ts                          # Zustand global UI state
│   ├─ types.ts                          # Mirrors Rust types
│   └─ features/
│      ├─ TopBar.tsx                     # Top-of-window: add folder, rescan,
search, sort
│      ├─ Sidebar.tsx                    # Left: library / tag filters
│      ├─ ImageGrid.tsx                  # Virtualised file grid
│      ├─ DetailsPanel.tsx               # Right: SingleDetails / MultiDetails
(Title / Tags / Format / Info)
│      ├─ TagInput.tsx                   # Tag entry with autocompletion
│      ├─ Thumbnail.tsx                  # <img> wrapper with async src
│      └─ StatusBar.tsx                  # Bottom: scan progress, app version
│
├─ src-tauri/                            # Rust backend (Cargo package)
│   ├─ Cargo.toml
│   ├─ tauri.conf.json                   # App config: identifier, security, window
│   ├─ build.rs
│   ├─ capabilities/
│   │  └─ default.json                   # Capability manifest (asset scopes, etc.)
│   ├─ examples/
│   │  ├─ dump_meta.rs                   # Diagnostic: dump everything about a photo
│   │  └─ dump_tag_usage.rs              # Diagnostic: find photos tagged X
│   ├─ tests/
│   │  └─ metadata_fs.rs                 # Integration tests for metadata pipeline
│   └─ src/
│      ├─ main.rs                       # Windows subsystem entry — calls lib::run
│      ├─ lib.rs                        # Tauri builder, logging, handler
registration
│      ├─ error.rs                      # AppError, AppResult
│      ├─ types.rs                      # Rust-side IPC types (mirror src/types.ts)
│      ├─ db/
│      │  ├─ mod.rs                     # pack_global_id / unpack_global_id helpers
│      │  ├─ registry.rs                # Central registry.db schema + queries
│      │  ├─ library.rs                 # Per-folder library.db schema + queries
│      │  ├─ pool.rs                    # LibraryPool: registry + ATTACHed
libraries
│      │  ├─ search.rs                  # Cross-folder query builders (UNION ALL)
│      │  └─ legacy_migration.rs        # One-shot import of pre-redesign central
DB
│      ├─ core/
│      │  ├─ mod.rs                     # AppServices, FormatRegistry, dirs
│      │  ├─ scanner.rs                 # Parallel folder scan (writes rel_paths)
│      │  ├─ thumbnail.rs               # Thumb generation + caching (keyed by gid)
│      │  ├─ formats/                   # Per-format handlers, all read-only
```

```
│      │   │   ├── mod.rs                  # FormatHandler trait + FormatRegistry
│      │   │   ├── common.rs               # EXIF/dims utilities, verbatim-path
helpers
│      │   │   ├── xmp_packet.rs           # XMP parser (read only)
│      │   │   ├── win_shell.rs            # Windows IPropertyStore reader
│      │   │   ├── jpeg.rs
│      │   │   ├── png.rs
│      │   │   ├── webp.rs
│      │   │   ├── gif.rs
│      │   │   ├── tiff.rs
│      │   │   └── stubs.rs                # HEIC, PDF, video, RAW, …
│      │   └── metadata/
│      │       ├── mod.rs
│      │       ├── read.rs                 # Delegates to registry + Shell + sidecar
│      │       └── sidecar.rs              # Legacy `.xmp` reader (never writes)
│      └── commands/                       # Tauri command handlers
│          ├── mod.rs
│          ├── library.rs                  # add/remove/list folders, rescan
│          ├── images.rs                   # query, get, update, batch_update, delete
│          ├── tags.rs                     # list / rename / delete tags
│          ├── collections.rs              # smart collections (skeleton in v1)
│          ├── thumbs.rs                   # thumb + asset path resolvers
│          └── diag.rs                     # log_frontend bridge
│
├── scripts/
│   ├── screenshot-app.ps1                # Verify UI screenshot
│   ├── screenshot-scrolled.ps1           # Screenshot with programmatic scroll
│   └── test-multiselect-tag.ps1          # E2E UI test skeleton
│
├── docs/                                 # This documentation
│   ├── user-manual/                      # mdBook: user-facing manual
│   ├── developer/                        # mdBook: this guide
│   ├── shared/                           # Shared CSS / JS for both books
│   ├── index.html                        # Landing page linking both docs
│   └── build.ps1                         # HTML + PDF build script
│
├── index.html                            # Vite entry
├── vite.config.ts
├── tailwind.config.js
├── postcss.config.js
├── tsconfig.json
├── package.json
├── README.md
└── .gitignore
```

What lives where

- **Anything user-visible** (button text, layout, animations) is under `src/`.

- **Anything touching disk** (files, DB) is under `src-tauri/src/`. If you find frontend code doing FS work, that's a bug.
- **Anything that runs at build time** (Tauri config, capabilities, Cargo settings) is under `src-tauri/` outside `src/`.
- **Documentation source** is under `docs/`. Generated HTML lands in `docs/user-manual/book/` and `docs/developer/book/`; PDFs land in `docs/`.

Backend modules

`lib.rs` — app entry

Responsibilities:

- Initialise file-based logging (`init_logging`).
- Build `AppServices { pool, thumb_cache_dir, app_data_dir, formats }` under `Arc`.
- Register every Tauri plugin (`dialog`, `opener`) and command handler in `invoke_handler!`.
- Manage the app run loop.

Notable pitfalls:

- `init_logging` **must not** call `tracing_subscriber::fmt().init()` if a Tauri plugin (like the removed `tauri-plugin-log`) already set a global subscriber. The prior implementation crashed with `PluginInitialization("log", "attempted to set a logger after…")`; the current version owns the logger outright.

`types.rs` — IPC types

- `LibraryFolder` — one registered folder; includes `isAvailable`.
- `ImageSummary`, `ImageDetails` — full and abridged views of an image row. `id` is a packed global ID.
- `MetadataPatch` — the "what to change" payload for `update_image_metadata` and `batch_update_metadata`.
- `SyncRiskWarning` — non-null result of `check_folder_sync_risk`.
- `double_option` module — custom Serde deserializer for `Option<Option<T>>`. Distinguishes "field missing" from "field explicitly null" on the wire.

Every struct uses `#[serde(rename_all = "camelCase")]` so JSON keys are camelCase but Rust fields stay snake_case.

`error.rs`

`AppError` enum (via `thiserror`):

- `Io(std::io::Error)` — filesystem errors.
- `Db(rusqlite::Error)` — DB errors.
- `Pool(String)` — LibraryPool / mutex-poisoning errors.
- `MetadataRead(String)` — user-facing metadata read failures.
- `Scan(String)` — scanner-specific issues.
- `Internal(String)` — anything else.

All commands return `AppResult<T>` (= `Result<T, AppError>`). The `Display` impl for `AppError` is safe to surface to the user (no absolute paths in strings without context).

db/ — two-tier storage

See Database schema and Database redesign.

- `db/mod.rs` — packed global ID helpers.
- `db/registry.rs` — central `registry.db` schema (`library_folders`, `smart_collections`, `app_settings`) plus insert/list/update/delete helpers.
- `db/library.rs` — per-folder `library.db` schema (`folder_meta`, `images`, `tags`, `image_tags`, `images_fts`) plus `upsert_image`, `set_image_meta`, `apply_metadata_patch`, `rename_tag`, `delete_tag`, and FTS-rebuild helpers. Uses `LibraryDb::lock() -> MutexGuard<'_, Connection>` for writes.
- `db/pool.rs` — `LibraryPool`. Owns the registry connection with every library `ATTACH DATABASE`-ed on it, plus a lazy map of per-folder writer connections. `LibraryPool::with_registry`, `LibraryPool::library(folder_id)`, `LibraryPool::add_folder`, `LibraryPool::remove_folder`.
- `db/search.rs` — cross-folder query builders (`query_images`, `list_all_tags`, `get_image_by_gid`, `apply_metadata_patch_by_gid`, `group_gids_by_folder`).
- `db/legacy_migration.rs` — one-shot importer for the old central `library.db`.

`core/scanner.rs`

Pipeline for `add_library_folder` and `rescan_*`:

1. **Walk.** `jwalk::WalkDir` traverses the folder in parallel.

2. **Filter.** Extensions checked against `IMAGE_EXTS`. `.magpie/library.db` (and WAL/SHM) is excluded from the walk.
3. **Diff.** For each file, look up by *folder-relative* path in `images`; skip if `mtime_ms` matches.
4. **Extract.** In parallel via `rayon`: EXIF read, XMP read (native handlers), Windows Shell property read (formats without native parsers), content hash (`XXH3`).
5. **Upsert.** DB writes are serialised via the folder's single `LibraryDb` mutex; scanner buffers batches to amortise txn overhead.
6. **Thumbnails.** Enqueue small + medium keyed by packed global ID.
7. **Progress.** Emit `app://scan { folder_id, done, total, current }`.

Scanning is bounded by disk read speed on cold cache and by CPU on warm cache.

`core/thumbnail.rs`

```
pub fn ensure_thumbnails(cache_dir: &Path, src: &Path, gid: i64) -> Result<()>;
pub fn thumb_path(cache_dir: &Path, gid: i64, size: ThumbSize) -> PathBuf;
pub fn delete_thumbnails(cache_dir: &Path, gid: i64);
```

Decode via `image::open`, resize via `fast_image_resize` (SIMD Lanczos3), encode via `webp::Encoder` at quality 80. `gid` is the packed global ID (`folder_id * 1_000_000_000 + local_id`), so thumbnails for different folders never collide.

`core/metadata/read.rs`

`read_all(registry, path) -> ImageMetaFromFile` runs the read pipeline:

1. Handler's `read_technical` for dimensions + EXIF-derived fields.
2. Handler's `read_user` for XMP-parseable formats.
3. Windows Shell property store fallback for RAW / HEIC / MP4 / PDF.
4. Legacy `.xmp` sidecar read for backward compatibility (no writes).

Non-fatal errors are collected into a per-file warning log; the scanner continues on the next file.

The full pipeline is described in Metadata read path.

`core/formats/`

Each supported extension is owned by one `FormatHandler` implementation, and every handler is now **read-only** (see Database redesign):

- `mod.rs` — declares the `FormatHandler` trait (`name`, `extensions`, `kind`, `read_technical`, `read_user`), `TechnicalMeta`, `UserMeta`, `FormatKind`, and the `FormatRegistry` that lives on `AppServices`.
- `xmp_packet.rs` — hand-written streaming XMP parser (no writer). Extracts packets containing title, subjects, description, and Microsoft-Photo keywords.
- `common.rs` — shared helpers: EXIF → technical metadata, dimensions, verbatim-prefix stripping.
- `jpeg.rs`, `png.rs`, `webp.rs`, `gif.rs`, `tiff.rs` — native readers.
- `stubs.rs` — HEIF, video, PDF, RAW, BMP/EXR/HDR/SVG readers. These lean on `imagesize` for dimensions and defer user metadata to `win_shell::read_user_meta`.
- `win_shell.rs` — Windows-only Shell property store **reader**.

No `metadata/write.rs`

The write path is gone. `FormatHandler` has no `write_user`; `common::atomic_write_bytes`, `win_shell::write_user_meta`, `xmp_packet::build_xmp_packet`, and `xmp_packet::merge_user_edits` were all removed. Every user-metadata mutation now goes through `db::library::apply_metadata_patch` and lives in the folder's `library.db`.

`commands/*.rs`

Each file exposes 3–5 `#[tauri::command]` async functions. See Tauri command reference for the full list.

Common patterns:

- `services: State<'_, Arc<AppServices>>` for shared state.
- `app_handle: AppHandle` for emitting events.
- Return `AppResult<T>`; Serde does the JSON legwork.
- Tracing spans on entry so `app.log` shows every IPC hit.

Global IDs are unpacked at the command boundary — commands call `db::unpack_global_id(id)` (or the higher-level `search::get_image_by_gid` /

`search::apply_metadata_patch_by_gid`) so per-folder logic never sees the packed form.

Frontend modules

The frontend is intentionally small — the goal is to be a translation layer between UI events and Tauri commands, not to hold domain logic.

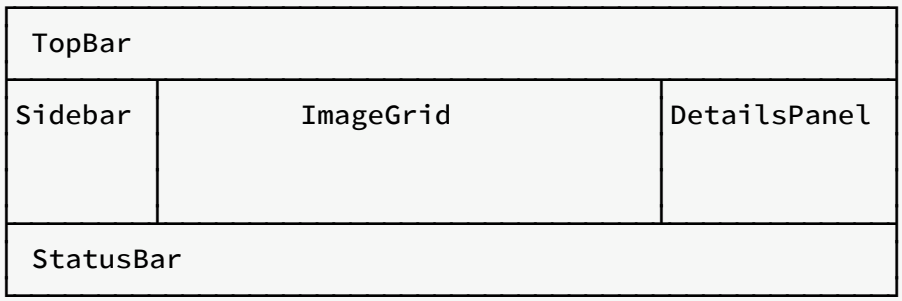
`src/main.tsx`

Boots React. Wraps `<App />` in a `QueryClientProvider` with a single QueryClient configured for:

- `staleTime: 0` — refetch on refocus.
- `retry: 1` — one retry on network / IPC errors.
- Query cache keys convention (below).

`src/App.tsx`

Owns the top-level layout:



Also hosts the global keyboard listener (Delete key), and mounts event listeners for `app://image-updated` and `app://images-deleted` — both of which invalidate the relevant query keys.

`src/store.ts` (Zustand)

Global UI-only state:

```
interface Store {
  view: View                // 'all' | { folderId } | { tag }
  search: string
  sort: ImageSort
  extraFilter: ImageFilter  // sidebar filters composed
  selection: { ids: Set<number>; anchor: number | null }
  detailsOpen: boolean
  // setters
  setView, setSearch, setSort, setSelection, clearSelection, …
}
```

No persistence in v1 — reloading the app resets to `view: 'all'`, empty search, no selection.

`filterFromView(view, extra, search)` composes the sidebar view, the extra filter, and the search string into a single `ImageFilter` sent to `query_images`.

src/ipc.ts

Thin, typed wrappers around `invoke`:

```
export const updateImageMetadata = (id: number, patch: MetadataPatch) =>
  invoke<ImageDetails>('update_image_metadata', { id, patch })

export const batchUpdateMetadata = (ids: number[], patch: MetadataPatch) =>
  invoke<number[]>('batch_update_metadata', { ids, patch })

// …
```

Plus event-listener helpers:

```
export const onImageUpdated = (h: (id: number) => void) =>
  listen<number>('app://image-updated', e => h(e.payload))
```

And the diagnostic helper `logFrontend(level, msg)` for pushing crumbs into the backend log.

src/types.ts

Every IPC struct duplicated from Rust, kept in sync manually. See IPC boundary for the rationale (no derived types).

src/features/

TopBar.tsx

- Add folder button (opens Tauri dialog, calls `add_library_folder`, invalidates `['folders']` and `['images']`).
- Rescan button (calls `rescan_all`).
- Live search input (debounced 200 ms → `useStore.setSearch`).
- Sort dropdown + direction toggle.
- Toggle-details icon.

Sidebar.tsx

- `['folders']`, `['tags']` queries fed to three collapsible sections.
- Click handlers set `useStore.view` and clear other filters.
- Right-click context menu for `Remove folder`, `Rescan folder`, `Rename tag`, `Delete tag`.

ImageGrid.tsx

- `useQuery(['images', filter, sort, page])` returns paginated results.
- `useVirtualizer` from TanStack Virtual computes visible rows.
- Each tile is a `Thumbnail` component; `Ctrl / Shift / plain` click logic mutates `useStore.selection`.

DetailsPanel.tsx

- Reads `useStore.selection` and dispatches:
  - 0 selected → placeholder.
  - 1 selected → `<SingleDetails id=… />`.
  - 1 selected → `<MultiDetails ids=… />`.

`SingleDetails` renders four fixed sections:

1. **Title** — editable input, auto-saves via `updateImageMetadata`.

2. **Tags** — `TagInput`, auto-saves via `updateImageMetadata`.
3. **Format metadata** — read-only: handler name. Room to grow into per-format editable fields (GPS, description, …) as those handlers grow their surface area.
4. **File info** — `<dl>` of the `technical` list returned by the backend plus filename, size, format, mtime, and import time.

Local edit state is seeded from the query result only when the `id` changes (guarded by `lastLoadedId.current`), avoiding the "stomp typing on refetch" bug.

`MultiDetails` shows two `TagInput`s (Add / Remove) and an Apply button. It uses refs (`tagsAddRef`, `tagsRemoveRef`) mirroring state so the mutation dispatch always sees the latest values, even if a blur-triggered `setTagsAdd` and a click on `Apply` land in the same React batch.

`TagInput.tsx`

- Controlled by parent's `tags: string[]` + `onChange(next: string[])`.
- Commits the current draft on Enter, Space, Comma, or blur.
- Autocompletion pulled from `list_tags(prefix)` via TanStack Query.
- Backspace on empty input pops the last tag.

`Thumbnail.tsx`

- Resolves `getThumbPath(id, 'small')` on mount, sets `<img src>` to the returned asset URL.
- Placeholder while the thumbnail is being generated (rare, only on first scan).

`StatusBar.tsx`

- Listens to `app://scan` events, shows a live progress bar and message.
- Static labels for app version, DB size, thumbnail cache size.

Query key conventions

Query key	Fetched by	Invalidated by
`['folders']`	`listLibraryFolders`	add/remove/rescan folder, delete images

Query key	Fetched by	Invalidated by
`['tags']`	`listTags`	any metadata update, tag rename/delete
`['tag', prefix]`	`listTags(prefix)`	any metadata update
`['images', filter, sort, page]`	`queryImages`	any image mutation
`['image', id]`	`getImage`	`update_image_metadata` (via `setQueryData`),
		`batch_update_metadata` (via `invalidateQueries`),
		`app://image-updated` event
`['smartCollections']`	`listSmartCollections`	create/delete collection

Styling

- Tailwind for utility classes.
- Global styles in `src/index.css`: dark background, rounded buttons, focus rings.
- No CSS-in-JS. Component-scoped styles are className concatenations via `clsx` when needed.

Database schema

Magpie's storage is **two-tier**: one small central registry DB per machine, one library DB per registered folder that lives inside the folder itself. See Database redesign for the motivation and cross-folder query strategy.

- **Registry DB** — `%APPDATA%\com.magpie.app\registry.db`. Owned schema in `src-tauri/src/db/registry.rs`.
- **Library DB** — `<folder>/.magpie/library.db`, one per registered folder. Owned schema in `src-tauri/src/db/library.rs`.

Registry DB (`registry.db`)

Small. Grows with the number of registered folders, not the number of files.

`library_folders`

Column	Type	Notes
id	INTEGER	PK, autoincrement.
path	TEXT	Absolute, canonical, UNIQUE.
added_at	INTEGER	Unix ms.
last_scan_at	INTEGER	Unix ms; NULL before first scan finishes.
is_available	INTEGER	0/1. 0 = `.magpie/library.db` couldn't be attached.

`smart_collections`

Column	Type	Notes
id	INTEGER	PK, autoincrement.
name	TEXT	
filter	TEXT	JSON-serialised `ImageFilter`.

Column	Type	Notes
sort_order	INTEGER	

`app_settings`

Key-value store for app-wide preferences the UI persists across launches.

Column	Type	Notes
key	TEXT	PK.
value	TEXT	Free-form (JSON, plain string, …).

Library DB (`<folder>/.magpie/library.db`)

Fully self-contained: paths are folder-relative, tag names are stored inline, the FTS index sits next to the tags table. Copying the folder to a different disk/PC preserves everything.

`folder_meta` (singleton, row `id = 1`)

Column	Type	Notes
id	INTEGER	Constant `1`.
magpie_version	TEXT	`CARGO_PKG_VERSION` when the DB was created.
schema_version	INTEGER	Bump on breaking schema changes.
created_at	INTEGER	Unix ms.
last_scan_at	INTEGER	Nullable.

`images`

One row per file (of any format the registry recognises) scanned in this folder. The table is still called `images` for historical reasons; a future migration may rename it to `files`.

Column	Type	Notes
id	INTEGER	PK, autoincrement (local to this DB; global IDs are packed).
rel_path	TEXT	Folder-relative path, forward slashes, UNIQUE.
filename	TEXT	Basename (`IMG_1234.jpg`).
ext	TEXT	Lowercase extension, no dot.
size_bytes	INTEGER	From `fs::Metadata::len()` .
mtime_ms	INTEGER	Modification time in Unix ms.
width	INTEGER	Nullable; from handler's technical read.
height	INTEGER	Nullable.
content_hash	TEXT	Nullable; XXH3 128-bit hex digest.
taken_at	INTEGER	Nullable; Unix ms from EXIF `DateTimeOriginal` .
camera_make	TEXT	Nullable.
camera_model	TEXT	Nullable.
title	TEXT	Nullable; user-editable.
imported_at	INTEGER	Unix ms when the row was first inserted.
missing	INTEGER	0/1. 1 after a scan that failed to see the file.

Indexes: `idx_images_taken_at` , `idx_images_filename` , `idx_images_ext` .

`tags`

Column	Type	Notes
id	INTEGER	PK, autoincrement.
name	TEXT	UNIQUE COLLATE NOCASE.

`image_tags`

Many-to-many join.

Column	Type	Notes
image_id	INTEGER	FK → `images(id)` ON DELETE CASCADE.
tag_id	INTEGER	FK → `tags(id)` ON DELETE CASCADE.
PK		(image_id, tag_id).

Index: `idx_image_tags_tag(tag_id)`.

`images_fts` (virtual, FTS5)

```
CREATE VIRTUAL TABLE images_fts USING fts5(
    title, filename, tags,
    content='',
    contentless_delete=1,
    tokenize='unicode61 remove_diacritics 2'
);
```

- Contentless (`content=''`) — `rowid` is `images.id`; we join.
- `contentless_delete=1` — required for our rebuild-per-row strategy.
- Diacritics folded (`naive` = `naïve`).

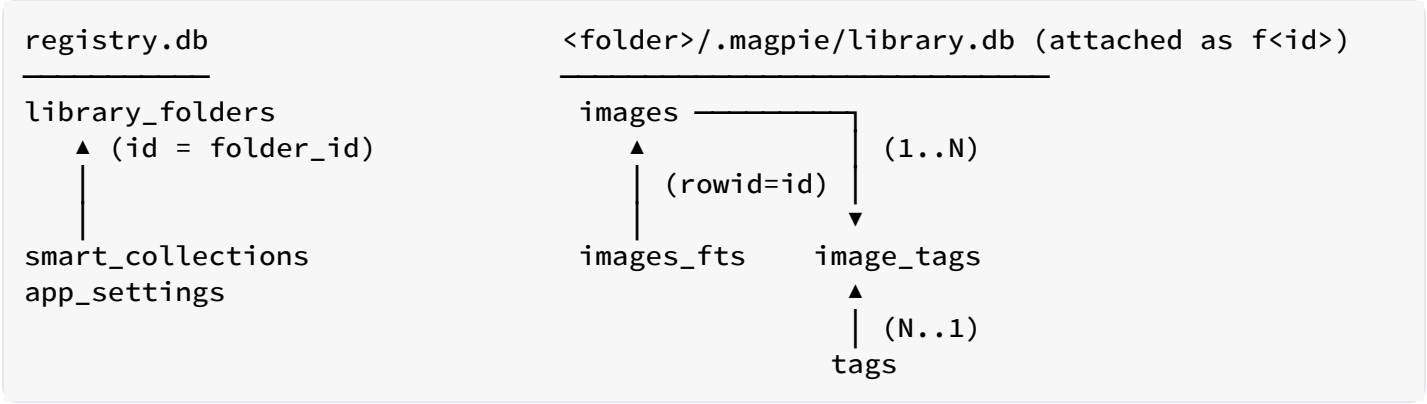
Global ID scheme

The IPC layer always exposes a packed *global* ID so the frontend never sees the per-folder `images.id` directly:

```
gid = folder_id * 1_000_000_000 + local_id
```

Room for 9 million folders × 1 billion images each, all inside JavaScript's `Number.MAX_SAFE_INTEGER`. Helper functions live in `src-tauri/src/db/mod.rs` (`pack_global_id`, `unpack_global_id`).

Relationships



Migrations

Registry: freshly created on first launch, no migration history yet.

Library: `folder_meta.schema_version` is bumped when the schema changes. Only version 1 exists today.

Legacy migration: on first launch after the redesign, if `registry.db` is missing but a legacy central `library.db` is present, `db::legacy_migration::migrate_legacy_central_db` splits the central DB into per-folder DBs, populates `registry.db`, and renames the legacy file to `library.db.migrated-<timestamp>` so nothing is destroyed. See Database redesign.

Tauri command reference

Every command below is registered in `src-tauri/src/lib.rs` inside `tauri::generate_handler![…]`. Signatures use `AppResult<T>` and camelCase JSON on the wire.

> **IDs are packed.** Every `id: i64` an image command accepts or returns is the *global* packed ID (`folder_id * 1_000_000_000 + local_id`). The frontend never sees per-folder local IDs directly.

Library management

`add_library_folder`

```
async fn add_library_folder(
    services: State<'_, Arc<AppServices>>,
    app_handle: AppHandle,
    path: String,
) -> AppResult<LibraryFolder>;
```

Canonicalises the path, inserts a row into `registry.db`, creates the folder's `.magpie/library.db`, attaches it on the registry connection, and spawns a background scan. Emits `app://scan` progress events.

`remove_library_folder`

```
async fn remove_library_folder(
    services: State<'_, Arc<AppServices>>,
    id: i64,
) -> AppResult<()>;
```

Detach the library from the registry connection and delete the `library_folders` row. **The `.magpie/library.db` file on disk is left in place** — it's the user's data and Magpie doesn't own the folder.

`list_library_folders`

```
async fn list_library_folders(
    services: State<'_, Arc<AppServices>>,
) -> AppResult<Vec<LibraryFolder>>;
```

Includes `isAvailable` — `false` when the folder's library couldn't be reached (removable drive unplugged, network share unreachable).

`rescan_folder`

```
async fn rescan_folder(
    services: State<'_, Arc<AppServices>>,
    app_handle: AppHandle,
    id: i64,
) -> AppResult<ScanResult>;
```

Re-walks the specified folder. Incremental: only files with a newer `mtime` than what's in that folder's DB get re-processed.

`rescan_all`

```
async fn rescan_all(
    services: State<'_, Arc<AppServices>>,
    app_handle: AppHandle,
) -> AppResult<Vec<ScanResult>>;
```

Rescan every *available* folder sequentially.

`check_folder_sync_risk`

```
async fn check_folder_sync_risk(
    path: String,
) -> AppResult<Option<SyncRiskWarning>>;
```

Non-null when `path` looks like it lives on a cloud-synced disk (OneDrive / Dropbox / Google Drive / iCloud / Box) or a UNC network share. The frontend calls this **before** `add_library_folder` and shows a `confirm` dialog with the returned message.

`SyncRiskWarning { provider: String, message: String }`.

Images

`query_images`

```
async fn query_images(
    services: State<'_, Arc<AppServices>>,
    filter: Option<ImageFilter>,
    sort: Option<ImageSort>,
    page: Option<Pagination>,
) -> AppResult<Page<ImageSummary>>;
```

Cross-folder query. Under the hood: `UNION ALL` across every attached library DB, then order + paginate. Each result row's `id` is packed with its `folder_id`.

`get_image`

```
async fn get_image(
    services: State<'_, Arc<AppServices>>,
    id: i64,
) -> AppResult<ImageDetails>;
```

Return full details for one image. **Side effect:** if the source file's `mtime` is newer than what's stored in the row, the format handler is asked to re-read user metadata (title + tags from XMP/`System.Keywords`) so that a Lightroom / Explorer edit made after import is picked up on next load. Fresh reads only update the row if the mtime moved forward; there is no periodic polling.

`update_image_metadata`

```
async fn update_image_metadata(
    services: State<'_, Arc<AppServices>>,
    app_handle: AppHandle,
    id: i64,
    patch: MetadataPatch,
) -> AppResult<ImageDetails>;
```

Apply a metadata patch to the folder's `library.db` (title, tags, `tags_add`, `tags_remove`). **The source file is never touched.** Rebuilds the FTS row so subsequent search reflects the change. Emits `app://image-updated`.

`batch_update_metadata`

```
async fn batch_update_metadata(
    services: State<'_, Arc<AppServices>>,
    app_handle: AppHandle,
    ids: Vec<i64>,
    patch: MetadataPatch,
) -> AppResult<Vec<i64>>;
```

Same as `update_image_metadata` but for many photos. `ids` are packed globals; the command groups them by folder and touches each folder's `library.db` once inside a single transaction. Returns the list of `ids` that succeeded; failures are logged and skipped.

`delete_images`

```
async fn delete_images(
    services: State<'_, Arc<AppServices>>,
    app_handle: AppHandle,
    ids: Vec<i64>,
    permanent: Option<bool>,
) -> AppResult<DeleteResult>;
```

Move to Recycle Bin (default) or permanently delete. Also removes the DB rows and cached thumbnails. Returns per-file success/failure.

Tags

`list_tags`

```
async fn list_tags(
    services: State<'_, Arc<AppServices>>,
    prefix: Option<String>,
) -> AppResult<Vec<TagStats>>;
```

Aggregates tags across every attached library via `UNION ALL` grouped by `name COLLATE NOCASE`.

`rename_tag`

```
async fn rename_tag(
    services: State<'_, Arc<AppServices>>,
    old_name: String,
    new_name: String,
) -> AppResult<()>;
```

Global rename across every available folder. Rebuilds FTS rows for every affected image. **Source files are not touched.**

`delete_tag`

```
async fn delete_tag(
    services: State<'_, Arc<AppServices>>,
    name: String,
) -> AppResult<()>;
```

Remove a tag from every folder's library. **Source files are not touched.**

Smart collections

`list_smart_collections`

```
async fn list_smart_collections(
    services: State<'_, Arc<AppServices>>,
) -> AppResult<Vec<SmartCollection>>;
```

`create_smart_collection`

```
async fn create_smart_collection(
    services: State<'_, Arc<AppServices>>,
    name: String,
    filter: ImageFilter,
) -> AppResult<SmartCollection>;
```

`delete_smart_collection`

```
async fn delete_smart_collection(
    services: State<'_, Arc<AppServices>>,
    id: i64,
) -> AppResult<()>;
```

Smart collections live in `registry.db` (they're app-wide, not per-folder).

Thumbnails and image paths

`get_thumb_path`

```
async fn get_thumb_path(
    services: State<'_, Arc<AppServices>>,
    id: i64,
    size: ThumbSize,
) -> AppResult<String>;
```

Return the absolute path of a cached thumbnail; generate on demand if missing. Thumbnails are indexed by the *packed global ID* so they don't collide between folders.

`get_image_path`

```
async fn get_image_path(
    services: State<'_, Arc<AppServices>>,
    id: i64,
) -> AppResult<String>;
```

Absolute path of the source image (folder root + `rel_path`), for the DetailsPanel inline preview via `convertFileSrc`.

Diagnostics

`log_frontend`

```
fn log_frontend(level: String, msg: String) -> AppResult<()>;
```

Push a log line into `app.log` from the renderer. `level` is one of `debug|info|warn|error`. Emitted with `target="frontend"` for easy filtering.

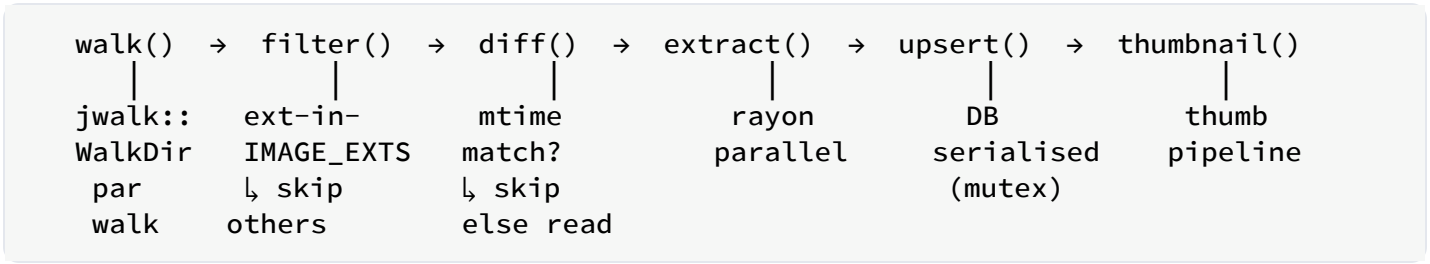
Scanner algorithm

Objective

Given a library folder path, populate the folder's per-folder `library.db` (`<folder>/.magpie/library.db`) with a row per supported file, extract metadata, generate thumbnails, and keep the DB in sync with the filesystem on subsequent runs — all while staying responsive.

Every path stored in the DB is **folder-relative** (forward slashes). The scanner joins root + relative path when it needs the actual filesystem location.

Pipeline



Progress events (`app://scan { folder_id, done, total, current }`) are emitted every N files (default 20).

Stage 1 — walk

`jwalk::WalkDir::new(root).parallelism(Parallelism::RayonNewPool(n))` walks the folder tree in parallel across all cores, streaming `DirEntry` values. Symlinks are followed (with cycle detection); hidden files are respected per OS convention.

Stage 2 — filter

An entry is kept iff:

- It's a regular file (not a directory, socket, or device).
- Its lowercase extension is in `core::IMAGE_EXTS`: `jpg jpeg jpe jfif jif png gif bmp tif tiff webp heic heif` `cr2 cr3 nef arw dng raf orf pef x3f`.
- Its filename doesn't start with `.` (dotfiles).

Discarded entries increment a counter used for the progress denominator but otherwise do no work.

Stage 3 — diff

For each kept entry, Magpie looks up the *folder-relative* path in the folder's `images` table:

```
SELECT id, mtime_ms FROM images WHERE rel_path = ?1
```

`.magpie/library.db` itself (and its WAL / SHM sidecar files) is excluded from the walk so the scanner doesn't try to import its own storage.

- **New file** (no row): full extract required.
- **Existing, mtime unchanged**: skip.
- **Existing, mtime changed**: partial re-extract (metadata + hash), leaving other columns intact.

The lookup is O(log n) thanks to the UNIQUE index on `images.path`.

Stage 4 — extract

For each file that needs work, the following runs on a rayon worker:

1. **Stat** — `fs::metadata(path)` for `size_bytes` and `mtime_ms`.
2. **Content hash** — stream the file through `XXH3_128`. On an SSD this is disk-bound at ~2 GB/s; on HDD it's the bottleneck.
3. **Handler lookup** — `registry.for_ext(ext)` picks the `FormatHandler` for the file's extension.
4. **Technical read** — the handler's `read_technical` produces ordered key/value pairs; those we recognise fold into `taken_at`, `camera_make`, `camera_model`, `width`, `height`.
5. **User meta** — `metadata::read::read_all` combines the handler's `read_user` with any legacy sidecar for `title` and `tags`.

Any per-file failure is logged (`WARN`) and doesn't abort the folder scan. The file gets a row with whatever metadata succeeded; missing fields are left NULL.

Stage 5 — upsert

The folder's `library.db` is the single writer for this folder; different folders write in parallel. Extracted rows are pushed to a bounded channel and a writer task commits batches of up to 128 rows per transaction:

```
let tx = conn.transaction()?;
for row in batch { library::upsert_image(&tx, &row)?; }
tx.commit()?;
```

The upsert SQL uses `rel_path` (folder-relative) as the conflict key:

```
INSERT INTO images (rel_path, filename, ext, size_bytes, mtime_ms,
                    width, height, content_hash, taken_at,
                    camera_make, camera_model, title, imported_at, missing)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, 0)
ON CONFLICT(rel_path) DO UPDATE SET
    size_bytes    = excluded.size_bytes,
    mtime_ms      = excluded.mtime_ms,
    width         = excluded.width,
    height        = excluded.height,
    content_hash  = excluded.content_hash,
    taken_at      = excluded.taken_at,
    camera_make   = excluded.camera_make,
    camera_model  = excluded.camera_model,
    title         = excluded.title,
    missing       = 0;
```

After the row lands, tags are reconciled: everything in `image_tags` for this image is deleted, then re-inserted from the fresh tag list. Both happen in the same transaction as the image upsert.

The FTS5 row is rebuilt via `rebuild_fts_row_tx` at the very end.

Stage 6 — thumbnails

Once a row is in the DB, the scanner enqueues a thumbnail task for that image id. Thumbnails run on the same rayon pool but with lower priority so they don't starve extraction.

`ensure_thumbnails(cache_dir, src_path, gid)`:

1. For each size (small, medium):
   - Path = `cache_dir / "<gid>-<size>.webp"` where `gid` is the *packed global ID* (`pack_global_id(folder_id, local_id)`).
   - Skip if the thumbnail's mtime ≥ the source's mtime.
2. Decode the source with `image::open`.
3. Resize with `fast_image_resize::Resizer` (Lanczos3, SIMD).
4. Encode with `webp::Encoder::new(&image).encode(80)`.
5. Write bytes atomically (temp + rename).

Failures at any stage are logged and don't affect the DB row.

Handling deletions

If a file that was in the DB is no longer found on disk during a scan, the scanner does **not** delete the row automatically — the `missing` flag in the `images` table exists to mark it (v1: not exposed in UI). A future contribution can add an explicit "Clean up missing photos" UI action.

Incremental performance

The diff step is what makes rescans fast:

- 50 000 photos, no changes: `~2 seconds` (mostly the DB lookup and a stat call per file).
- 50 000 photos, 100 new: `~5 seconds` (mostly hashing + XMP read for the 100).
- 50 000 photos, cold cache (thumbnails missing): `~2 minutes` (bounded by encoding).

Thumbnail pipeline

Sizes

Two cached sizes per image, both stored as WebP quality 80:

Enum name	Long-edge px	Used by
`Small`	~256	`ImageGrid` tiles (main use case).
`Medium`	~512	(Reserved for a future zoom-in mode.)
`Large`	~1024	(Currently only generated on request.)

Aspect ratio is preserved — the short edge is proportional. A 6000×4000 source becomes a 256×170 small and 512×341 medium.

Location

Files live in `%APPDATA%\com.magpie.app\thumbs\`:

```
<id>-small.webp
<id>-medium.webp
```

Using the image id (rather than a hash of the source path) keeps the cache invariant across file renames: as long as Magpie still knows about the image, the thumbnail is reusable.

Generation

```
pub fn ensure_thumbnails(cache_dir: &Path, src: &Path, image_id: i64) ->
Result<()> {
    let src_mtime = fs::metadata(src)?.modified()?;
    for size in &[ThumbSize::Small, ThumbSize::Medium] {
        let out = thumb_path(cache_dir, image_id, *size);
        if is_fresh(&out, src_mtime) { continue; }
        let img = image::open(src)?;
        let target = pick_target_size(&img, *size);
        let thumb = resize_rgba(&img.to_rgba8(), target);
        save_webp(&thumb, &out)?;
    }
    Ok(())
}
```

Details:

- `image::open` handles JPEG, PNG, GIF, TIFF, WebP, HEIC (with the `image` crate's HEIC feature enabled). For unsupported RAW formats, `open` returns an error and the thumbnail is skipped (grid renders a placeholder).
- `resize_rgba` uses `fast_image_resize::Resizer` with the `Lanczos3` filter. On x86_64 this dispatches to SSE4/AVX2 SIMD automatically.
- `save_webp` writes to `<out>.tmp` then renames — atomic, so a crash mid-encode never leaves a corrupt WebP.

Freshness

A thumbnail is considered fresh iff its file mtime is ≥ the source file's mtime. Editing metadata does not change the source mtime enough to invalidate a thumbnail (unless Magpie's embed-XMP write touched the JPEG — in which case the thumbnail regenerates on next scan, which is correct because the JPEG has actually changed).

Deletion

`delete_thumbnails(cache_dir, image_id)` is called from `delete_images`. It removes every size:

```
for size in [Small, Medium, Large] {
    let _ = fs::remove_file(thumb_path(cache_dir, image_id, size));
}
```

Errors are ignored — a missing thumb is the desired state.

Cache sizing

The `Small` WebP averages 4–8 KB per photo; `Medium` averages 16–32 KB. Total cache size on a 50 000-photo library is roughly 1–2 GB.

There's no automatic eviction in v1. Users who want to trim the cache can delete `%APPDATA%\com.magpie.app\thumbs\` — Magpie regenerates on demand.

Async model

`ensure_thumbnails` is CPU-bound (decode + resize + encode). It's called from Tauri commands via `tauri::async_runtime::spawn_blocking` so it doesn't stall a Tokio worker. The scanner enqueues thumb tasks on the rayon pool to overlap I/O with compute.

Metadata read path

Entry point

```
pub fn read_all(registry: &FormatRegistry, path: &Path) ->
AppResult<ImageMetaFromFile>;
```

Called from:

- The scanner (initial scan and rescans), to populate the folder's `library.db`.
- `get_image` (Tauri command), when the source file's `mtime` has moved forward since the row's cached `mtime_ms`.

`ImageMetaFromFile` is the union of everything we can pull from disk that the DB stores:

```
pub struct ImageMetaFromFile {
    pub title:        Option<String>,
    pub tags:         Vec<String>,      // deduped, in insertion order
    pub taken_at:     Option<i64>,      // ms since Unix epoch
    pub camera_make:  Option<String>,
    pub camera_model: Option<String>,
    pub width:        Option<u32>,
    pub height:       Option<u32>,
}
```

`ImageMetaFromFile` is used **only on import** (first scan of a file, or on mtime bump). After the row exists in the DB, subsequent edits stay in the DB via `library::apply_metadata_patch`.

Stages

1. Ask the handler

```
let handler = registry.for_ext(ext).ok_or(AppError::UnsupportedFormat)?;
let user      = handler.read_user(path)?;
let technical = handler.read_technical(path);
```

`read_user` returns the editable surface (title + tags). `read_technical` returns the ordered `TechnicalMeta` list used by the UI and by the scanner for dimensions / EXIF-derived fields (camera, taken_at). See File formats.

Each handler decides how to fetch this. Native handlers (JPEG, PNG, WebP, GIF, TIFF) use the shared `xmp_packet::parse_xmp` helper; stub handlers (HEIF, video, PDF, RAW, basic raster) implement `read_technical` from magic bytes and defer `read_user` to the Windows Shell fallback.

2. Windows Shell property store (import-only, Windows-only)

On Windows, if `handler.read_user` didn't return a title or tags (typical for RAW, HEIC, MP4, PDF), `win_shell::read_user_meta` is called. It uses `SHGetPropertyStoreFromParsingName` to open the file's shell property store read-only and asks for `System.Title`

- `System.Keywords`. This is how tags that a user typed into Explorer's *Properties → Details* dialog get imported into Magpie on first scan.

`win_shell` is strictly read-only after the redesign — there is no matching write path.

3. Legacy sidecar XMP

```
let sidecar = read_sidecar(&sidecar_path_for(path)).ok();
```

Sidecar path is `<file basename>.xmp`. Sidecars are a **legacy compatibility** path: Magpie no longer produces them, but older Magpie installs and other tools (Lightroom, digiKam) did. If a sidecar file exists it's parsed with the same XMP parser used for embedded packets and its fields are unioned into the result.

4. Merge and fold into DB row

`apply_user_meta` folds the handler's `UserMeta`, then the Windows Shell result, then the legacy sidecar (in that order — sidecar wins if it exists, matching Lightroom conventions on read). The merged `ImageMetaFromFile` is then written to the folder's `library.db` via `library::set_image_meta`.

XMP parser

`xmp_packet::parse_xmp(bytes) -> XmpUserMeta` is a hand-written streaming parser built on `quick_xml`. It normalises casing/namespaces because tools in the wild are notoriously inconsistent (`<X:XMPMETA>` , `<x:xmpmeta>` , and mixed-case in the same file).

Recognises both `dc:subject` (standard) and `MicrosoftPhoto:LastKeywordXMP` (Windows Explorer); unions the two sets, deduping case-insensitively while preserving the first observed casing. Handles both `<rdf:Alt>` and `<rdf:Seq>` inside `<dc:title>` ; the `x-default` language item wins if present, otherwise the first item.

FS-refresh check on `get_image`

Inside the `get_image` command, before returning:

```
if let Ok(fs_meta) = std::fs::metadata(&abs) {
    let disk_mtime = fs_meta.modified()?.duration_since(UNIX_EPOCH)?.as_millis()
as i64;
    if disk_mtime > row.mtime_ms {
        let fresh = read_all(&registry, &abs)?;
        library::set_image_meta(&mut conn, local_id, &fresh)?;
    }
}
```

Simple mtime comparison. If the file was modified after import, re-read metadata and overwrite the row. This is how an external tag edit (Windows Explorer, Lightroom export) becomes visible in Magpie on next click, without needing a full library rescan.

Note: the mtime bump wipes Magpie-side edits with whatever's currently in the file. In the new model, once the file is in the DB, edits should happen in Magpie. If you edit tags in Explorer after Magpie has been in charge, those Explorer edits win the next time `get_image` is called for that file — because the file's mtime is now newer, so Magpie re-reads it. Users who want to switch tagging tools mid-flight should be aware of this trade-off.

DB redesign: per-folder SQLite + central registry

Motivation

Prior to the `DB-redesign` branch, Magpie treated the file itself as the source of truth for user metadata. Titles and tags were embedded into JPEG/PNG/WebP/GIF via XMP and into every other format via the Windows Shell property system (`SHGetPropertyStoreFromParsingName`). The central SQLite database in `%APPDATA%\com.magpie.app\library.db` was just an index/cache used to make search fast.

That design had three sharp edges:

1. **Formats without a writable property handler couldn't be tagged at all.** BMP, DIB, SVG, EXR, HDR, PSD, JPEG 2000, several MPEG-TS variants — all locked out on Windows.
2. **Read/write parity across formats was hard.** JPEG got a hand-written XMP writer, PNG got `iTXt` , WebP got a RIFF chunk, GIF got the Application Extension trailer, and everything else went through Windows COM with its own bag of failure modes.
3. **Multi-folder libraries had to share one central DB.** Moving a folder to another PC, backing it up, or handing it to a colleague meant losing all the tagging work unless the user manually copied the central DB too.

The redesign flips the model: **the database is the sole source of truth for user metadata**, and each library folder gets its own DB sitting inside it as `<folder>/.magpie/library.db` .

Two-tier layout

```
Central registry DB (per-machine)          Per-folder library DBs (per-folder,
portable)
──────────────────────────────────         ──────────────────────────────────────

%APPDATA%\com.magpie.app\                  <folder1>\.magpie\library.db
    registry.db                            <folder2>\.magpie\library.db
    ├─ library_folders                     <folder3>\.magpie\library.db
    ├─ smart_collections                       ├─ folder_meta (singleton)
    └─ app_settings                            ├─ images (rel_path, ...)
                                               ├─ tags
                                               ├─ image_tags
                                               └─ images_fts (FTS5)
```

Registry DB

Lives at `%APPDATA%\com.magpie.app\registry.db`. Small — grows with the number of registered folders, not the number of files. Schema:

```sql
CREATE TABLE library_folders (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    path          TEXT NOT NULL UNIQUE,            -- absolute folder path
    added_at      INTEGER NOT NULL,
    last_scan_at  INTEGER,
    is_available  INTEGER NOT NULL DEFAULT 1       -- 0 = folder missing
(removable drive unmounted)
);

CREATE TABLE smart_collections (
    id         INTEGER PRIMARY KEY AUTOINCREMENT,
    name       TEXT NOT NULL,
    filter     TEXT NOT NULL,                      -- JSON-encoded ImageFilter
    sort_order INTEGER NOT NULL DEFAULT 0
);

CREATE TABLE app_settings (
    key   TEXT PRIMARY KEY,
    value TEXT NOT NULL
);
```

Library DB

Lives at `<folder>/.magpie/library.db`. Fully self-contained: every path is stored relative to `<folder>`, and tag names are stored inline (no reference to a global tag ID table). Copy the folder to another PC or archive it as a `.zip` and everything works.

```sql
CREATE TABLE folder_meta (
    id              INTEGER PRIMARY KEY CHECK (id = 1),
    magpie_version  TEXT NOT NULL,
    schema_version  INTEGER NOT NULL,
    created_at      INTEGER NOT NULL,
    last_scan_at    INTEGER
);

CREATE TABLE images (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    rel_path      TEXT NOT NULL UNIQUE,            -- relative to folder root,
forward slashes
    filename      TEXT NOT NULL,
    ext           TEXT NOT NULL,
    size_bytes    INTEGER NOT NULL,
    mtime_ms      INTEGER NOT NULL,
    width         INTEGER,
    height        INTEGER,
    content_hash  TEXT,
    taken_at      INTEGER,
    camera_make   TEXT,
    camera_model  TEXT,
    title         TEXT,
    imported_at   INTEGER NOT NULL,
    missing       INTEGER NOT NULL DEFAULT 0
);
CREATE INDEX idx_images_taken_at ON images(taken_at);
CREATE INDEX idx_images_filename ON images(filename);
CREATE INDEX idx_images_ext      ON images(ext);

CREATE TABLE tags (
    id   INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL UNIQUE COLLATE NOCASE
);

CREATE TABLE image_tags (
    image_id INTEGER NOT NULL REFERENCES images(id) ON DELETE CASCADE,
    tag_id   INTEGER NOT NULL REFERENCES tags(id)   ON DELETE CASCADE,
    PRIMARY KEY (image_id, tag_id)
);
CREATE INDEX idx_image_tags_tag ON image_tags(tag_id);

CREATE VIRTUAL TABLE images_fts USING fts5(
    title, filename, tags,
    content='',
    contentless_delete=1,
    tokenize='unicode61 remove_diacritics 2'
);
```

Global ID scheme

Every image row must have a *globally unique* ID that the frontend can pass through `getImage(id)`, invalidate queries by, etc. — but the per-folder DB only knows its own local IDs starting at 1.

Packed encoding:

```
pub const FOLDER_ID_MULT: i64 = 1_000_000_000;

pub fn pack(folder_id: i64, image_id: i64) -> i64 {
    folder_id * FOLDER_ID_MULT + image_id
}
pub fn unpack(global_id: i64) -> (i64, i64) {
    (global_id / FOLDER_ID_MULT, global_id % FOLDER_ID_MULT)
}
```

Room for **9 M folders × 1 B images each**, all inside JavaScript's `Number.MAX_SAFE_INTEGER` (2^{53} ≈ 9×10^{15}) so no BigInt gymnastics on the frontend.

Cross-folder search via `ATTACH DATABASE`

At startup the registry connection attaches every registered library DB:

```
ATTACH DATABASE 'C:\photos\2023\.magpie\library.db' AS f1;
ATTACH DATABASE 'C:\photos\2024\.magpie\library.db' AS f2;
```

Global queries become a `UNION ALL` across attached schemas, with the folder ID synthesised into each row's ID column at select time:

```
SELECT (1 * 1000000000 + id) AS gid, 1 AS folder_id, ... FROM f1.images WHERE
missing = 0
UNION ALL
SELECT (2 * 1000000000 + id) AS gid, 2 AS folder_id, ... FROM f2.images WHERE
missing = 0
```

FTS across libraries works identically — each library has its own `images_fts` and we `UNION` the matches.

SQLite allows up to 125 attached DBs per connection with `SQLITE_LIMIT_ATTACHED`, and we lift the limit from the default 10 in `LibraryPool::open`.

Attach/detach is amortized: it happens exactly once when a folder is added or removed, never per query. The registry connection lives for the app lifetime; the `LibraryPool` mediates access to the individual library DBs when we need a *single-folder* write (scanning, tag edits).

`LibraryPool`

New type in `src-tauri/src/db/pool.rs`:

```
pub struct LibraryPool {
    /// Registry-owning connection; every library DB is ATTACHed here for
    /// cross-folder reads. Guarded by a Mutex — search reads and folder
    /// add/remove serialize through it, which is fine because those are
    /// low-frequency.
    reg: Arc<Mutex<Connection>>,
    /// Per-folder writer connections. Populated lazily.
    libraries: RwLock<HashMap<i64, Arc<LibraryDb>>>,
    /// folder_id → filesystem path of that library DB, snapshot from the
    /// registry, kept in sync via add/remove_folder.
    library_paths: RwLock<HashMap<i64, PathBuf>>,
}
```

Read path (cross-folder queries):

- `LibraryPool::with_registry_conn(|conn| ...)` locks the registry mutex and hands the caller the attached connection. Callers build `UNION ALL` queries against `f1.images`, `f2.images`, …

Write path (single-folder edits, scanning):

- `LibraryPool::library(folder_id) -> Arc<LibraryDb>` returns a writer connection for that folder's DB, opening it if needed. All writes to a folder serialize through its own connection Mutex — different folders can be written in parallel.

Reading pre-existing embedded tags on first scan

Format handlers keep `read_user` and `read_technical`. The scanner still calls them for every file, so a Lightroom-tagged JPEG or a Windows-Explorer-tagged X3F imports its tags into the per-folder DB on the first scan. After that the DB is authoritative and the file bytes are never touched.

`FormatHandler::write_user` and `FormatHandler::can_write_tags` are removed from the trait. `win_shell::write_user_meta`, `win_shell::can_write_tags`, `common::atomic_write_bytes`, `xmp_packet::build_xmp_packet`, and `xmp_packet::merge_user_edits` are deleted along with the entire `core::metadata::write` module.

Migration from the legacy central DB

On `AppServices::new`, if `%APPDATA%\com.magpie.app\registry.db` does **not** exist but `library.db` (the old central DB) *does*, run the legacy migration:

1. Open the old DB read-only.
2. For every row in `library_folders`: a. `mkdir <folder>/.magpie` b. Create a fresh `library.db` there with the new schema. c. Copy `images` rows for that folder, translating absolute `images.path` → folder-relative `images.rel_path`. d. Copy `tags` for those images (via `image_tags`), preserving names. e. Rebuild `images_fts`.
3. Create the new `registry.db`, insert one `library_folders` row per migrated folder.
4. Rename the old file to `library.db.migrated-<yyyymmddThhmmss>` so nothing is destroyed.

Idempotent: skipped if `registry.db` already exists. Recoverable: if migration fails partway through, the legacy DB is still intact and the process retries on the next launch (registry.db is only written after every folder migrated successfully).

Sync-location warning

Because the library DB lives *inside* the folder, putting the folder in a sync location (OneDrive, Dropbox, Google Drive, iCloud) exposes the DB to concurrent-writer risk when two PCs edit tags simultaneously.

`commands::library::check_folder_sync_risk(path)` returns `Option<String>` — a human-readable warning if the path matches known sync-provider patterns or lives on a network share:

- Path contains `\OneDrive` (with or without `- <tenant>` suffix).
- Path contains `\Dropbox`.
- Path contains `\Google Drive` or `\GoogleDrive`.
- Path contains `\iCloudDrive` / `\iCloud Drive`.
- Path starts with `\\` (UNC / network share).

The frontend calls this before `add_library_folder` and shows a `confirm` dialog with the warning when non-null. The user can still proceed; Magpie just makes sure they know.

What the frontend sees change

- `ImageDetails` loses `writeMode`, `canWriteTags`, `metaWrittenAt` fields.
- No amber "library-only" hint under the tags editor — every file is now DB-taggable.
- `ImageSummary.id` is now a *packed global ID* (`folder_id * 1_000_000_000 + local_id`).
- `ImageSummary.path` is still the absolute path (registry + library join it back together for the UI, thumbnails, and inline preview).
- New IPC command `check_folder_sync_risk(path) -> Option<String>`.

What the file bytes see change

Magpie **never writes back into the source file** after this redesign.

- No XMP packets are inserted or updated.
- No Windows Shell IPropertyStore SetValue calls.
- No `.magpie.tmp` scratch files, no atomic rename dance for the source file.
- The file's `mtime` is not touched by Magpie unless the user explicitly deletes it via *Move to Recycle Bin*.

Existing embedded tags are read exactly once, at first scan, and imported into the per-folder DB. From then on the DB is the truth.

File formats

Magpie's metadata read path is powered by a small trait-based plug-in system rather than a hard-coded `match ext { … }`. Every handler is **read-only** — see Database redesign and Adding a format handler.

The `FormatHandler` trait

Every supported file type is represented by a Rust type that implements the trait declared in `src-tauri/src/core/formats/mod.rs`:

```
pub trait FormatHandler: Send + Sync {
    fn name(&self) -> &'static str;
    fn extensions(&self) -> &'static [&'static str];
    fn kind(&self) -> FormatKind;

    fn read_technical(&self, path: &Path) -> TechnicalMeta;
    fn read_user(&self, path: &Path) -> AppResult<UserMeta>;
}
```

- `name` — human-readable label shown in the details panel (e.g. `"JPEG"`).
- `extensions` — lowercase extensions this handler owns (`&["jpg", "jpeg"]`).
- `kind` — image / video / document / other. Used to classify entries and skip inappropriate operations (thumbnails, previews).
- `read_technical` — the read-only metadata shown at the bottom of the details panel. An ordered list of `(label, value)` pairs.
- `read_user` — extract title + tags from the file (XMP, EXIF, container-specific atoms). Called on import.

The `FormatRegistry`

`FormatRegistry` maps each lowercase extension to exactly one handler. It's constructed once at startup, wrapped in `Arc`, and carried by `AppServices`.

Handlers register themselves in `FormatRegistry::new()`. The registry is the sole authority for:

- **Which extensions are scannable.** `scanner` calls `registry.for_ext(ext).is_some()` to decide whether to index a file.

- **Where technical/user metadata comes from.** `metadata::read::read_all` builds an `ImageMetaFromFile` by calling `handler.read_technical(path)` and `handler.read_user(path)`, then merging in the Windows Shell property store result and any legacy `.xmp` sidecar.

First-scan tag import

On the *first* time the scanner sees a file, `read_user` and the Windows Shell fallback are consulted so tags that a previous tagger (Lightroom, Adobe Bridge, Explorer's *Properties → Details*) already put in the file are imported into the folder's `library.db`. After that, the DB is the source of truth and the file is never read again for user metadata unless its `mtime` moves forward.

Handler catalogue

The "Reads tags" column describes what the native handler can pull out during a first-scan import. Formats whose native handler can't parse tags still get scanned into the DB — the Windows Shell fallback usually fills in title/keywords on Windows.

Handler	Kind	Extensions	Reads tags?	Notes
JPEG	Image	`.jpg`, `.jpeg`	✅ XMP APP1	
PNG	Image	`.png`	✅ iTXt XMP	
WebP	Image	`.webp`	✅ RIFF XMP	
GIF	Image	`.gif`	✅ (GIF89a)	Adobe XMP-in-GIF Application Extension
TIFF	Image	`.tif`, `.tiff`	✅ tag 700	

Handler	Kind	Extensions	Reads tags?	Notes
HEIC / HEIF / AVIF	Image	`.heic`, `.heif`, `.hif`, `.avif`	❌ (Shell import)	Native read of box-level EXIF for `taken_at` only
JPEG XL	Image	`.jxl`	❌ (Shell import)	
JPEG XR / HD Photo	Image	`.jxr`, `.wdp`	❌ (Shell import)	
JPEG 2000	Image	`.jp2`, `.j2k`, `.j2c`, `.jpx`, `.jif`	❌ (Shell import)	
PSD	Image	`.psd`, `.psb`	❌ (Shell import)	
PDF	Document	`.pdf`	❌ (Shell import)	
MP4/MOV family	Video	`.mp4`, `.m4v`, `.mov`, `.3gp`, `.3gpp`, `.3g2`	❌ (Shell import)	
Matroska / WebM	Video	`.mkv`, `.mks`, `.mka`, `.webm`	❌ (Shell import)	
AVI	Video	`.avi`	❌ (Shell import)	
WMV/ASF	Video	`.wmv`, `.asf`	❌ (Shell import)	
MPEG-TS	Video	`.ts`, `.mts`, `.m2ts`	❌ (Shell import)	
Canon RAW	Image	`.cr2`, `.cr3`, `.crw`	❌ (Shell import)	
Nikon RAW	Image	`.nef`, `.nrw`	❌ (Shell import)	
Sony RAW	Image	`.arw`, `.sr2`, `.srf`, `.arq`	❌ (Shell import)	

Handler	Kind	Extensions	Reads tags?	Notes
Fuji RAW	Image	`.raf`	❌ (Shell import)	
Olympus RAW	Image	`.orf`, `.ori`	❌ (Shell import)	
Panasonic RAW	Image	`.rw2`, `.rwl`	❌ (Shell import)	
Pentax RAW	Image	`.pef`	❌ (Shell import)	
Samsung RAW	Image	`.srw`	❌ (Shell import)	
Sigma RAW	Image	`.x3f`	❌ (Shell import)	
Adobe/generic DNG	Image	`.dng`	✅ (TIFF-family)	
BMP	Image	`.bmp`, `.dib`	❌ (Shell import)	
OpenEXR	Image	`.exr`	❌	Library-only tags after import
Radiance HDR	Image	`.hdr`, `.hdp`	❌	Library-only tags after import
SVG	Image	`.svg`	❌	Library-only tags after import

"Shell import" means: the format's native `read_user` returns empty, and `metadata::read::read_all` then asks Windows' `SHGetPropertyStoreFromParsingName` for `System.Title` + `System.Keywords`. So on Windows the tag import still works for any format Explorer's *Details* pane recognises. On other platforms (future) these formats will simply start empty.

Where the code lives

```
src-tauri/src/core/formats/
├── mod.rs           # FormatHandler trait + FormatRegistry + TechnicalMeta /
UserMeta
├── common.rs        # Shared: EXIF → TechnicalMeta, dimensions, verbatim-prefix
helpers
├── xmp_packet.rs    # XMP parser (read-only)
├── win_shell.rs     # IPropertyStore fallback reader (Windows only; stub
elsewhere)
├── jpeg.rs          # native XMP APP1 reader
├── png.rs           # native iTXt XMP reader
├── webp.rs          # native RIFF XMP reader
├── gif.rs           # native Application-Extension XMP reader (GIF89a)
├── tiff.rs          # native tag 700 XMP reader
└── stubs.rs         # everything else, one struct per family
```

`stubs.rs` deliberately groups similar formats so adding a new “read technical + Shell-import user meta” format is one struct + one line in `FormatRegistry::new`.

Adding a format handler

Format handlers are the plug-in point for supporting a new file type. **All handlers are read-only** after the DB redesign — Magpie never writes back into source files. See Database redesign.

This page walks through the four-step recipe using a fictitious "MyFormat" (`.myf`) as an example.

1. Pick the right home

- Add a new struct to `src-tauri/src/core/formats/stubs.rs` if the handler only pulls technical metadata via `imagesize` / magic bytes, or
- Create a dedicated module `src-tauri/src/core/formats/myformat.rs` if you need a real parser (like `jpeg.rs` , `png.rs` , `tiff.rs`).

2. Implement `FormatHandler`

```rust
use super::common::{self, TechnicalMeta};
use super::{FormatHandler, FormatKind, UserMeta};
use crate::error::AppResult;
use std::path::Path;

pub struct MyFormat;

impl FormatHandler for MyFormat {
    fn name(&self) -> &'static str {
        "MyFormat"
    }
    fn extensions(&self) -> &'static [&'static str] {
        &["myf"]
    }
    fn kind(&self) -> FormatKind {
        FormatKind::Image
    }

    fn read_technical(&self, path: &Path) -> TechnicalMeta {
        let mut t = TechnicalMeta::default();
        common::append_file_basics(&mut t, path);
        // ... push more pairs specific to this format ...
        t
    }

    fn read_user(&self, path: &Path) -> AppResult<UserMeta> {
        // Native handlers can parse an embedded XMP packet here.
        // Stubs that rely on the Windows Shell fallback just
        // return an empty result and let read_all pick up the
        // slack.
        let _ = path;
        Ok(UserMeta::default())
    }
}
```

That's the whole trait. There's no `write_user`, no `can_write_tags`. If a file's format ever needs bespoke read-only handling of embedded tags (say, a proprietary MP4 atom), do it inside `read_user` — that's the only user-metadata seam we still expose.

3. Register it

Open `src-tauri/src/core/formats/mod.rs` and add the handler to `FormatRegistry::new`:

```rust
registry.register(Arc::new(myformat::MyFormat));
```

For a stub, add it to `stubs::register` (already called from `FormatRegistry::new`).

4. Write tests

Add a scanner-facing test that confirms:

- Files with the new extension are picked up by the walk.
- `handler.read_technical(path)` returns sensible values for a minimal fixture.
- `handler.read_user(path)` returns the tags Magpie should import on first sight (if the format supports embedded metadata).

The `registry_recognises_every_expected_extension` test in `src-tauri/tests/format_registry.rs` is the right place to prove the extension is known.

5. Update the docs

- Add a row to the handler catalogue on File formats.
- Add the extension to the user-manual Supported file formats page.

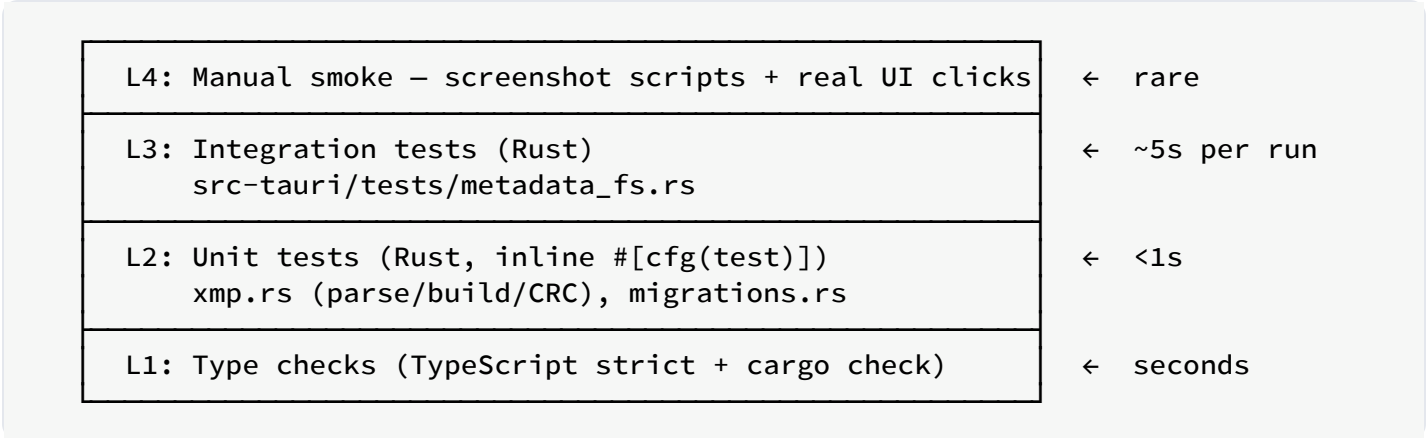
Design invariants

While writing a new handler, keep these invariants in mind:

- **Never write to the source file.** The DB is the source of truth for tags and titles. Any hypothetical need to write back should be discussed as an architectural change, not sneaked into a handler.
- **No blocking I/O on hot paths.** Handlers are invoked from the scanner (parallel) and the `get_image` command (foreground); restrict work in `read_technical` to what's cheap enough to run during a scan.
- **Failures are non-fatal.** If reading a specific field fails, return an empty `UserMeta` / partial `TechnicalMeta` — the caller will still insert the row with whatever succeeded.

Testing strategy

Layers



L1 — type checks

- `cargo check --workspace` for the Rust side (no unused warnings allowed in CI).
- `tsc -b --pretty false` for the frontend; strict mode enabled in `tsconfig.json` (`noUncheckedIndexedAccess`, `noImplicitAny`, `strictNullChecks`).

Both are cheap and run before every commit.

L2 — Rust unit tests

Colocated with modules in `#[cfg(test)] mod tests { … }`. Examples:

- `core/metadata/xmp.rs` — parser tests for Windows Explorer tags, Microsoft-only keyword lists, case variants, attribute-form values. Also covers the JPEG APP1 walker and (in integration tests) the PNG iTXt CRC.
- `db/queries.rs` — smoke tests for FTS row rebuild.

Run with:

```
cd src-tauri
cargo test --lib
```

L3 — Rust integration tests

`src-tauri/tests/metadata_fs.rs` exercises the full pipeline on temporary directories:

Test	Verifies
`read_sidecar_end_to_end`	Legacy sidecar read: title + tags.
`read_sidecar_case_variants`	XMP parser is namespace/case insensitive.
`fts_delete_after_tag_update_works`	The FTS5 `contentless_delete` regression is fixed.
`batch_tag_add_persists_for_every_image`	Bulk `tags_add` / `tags_remove` semantics.
`embed_xmp_roundtrip_jpeg`	JPEG embed writes, re-reads, replaces on second write.
`embed_xmp_roundtrip_png`	PNG iTXt embed, no chunk stacking on rewrite.
`embed_xmp_roundtrip_webp`	WebP RIFF `XMP ` chunk roundtrip, no stacking.
`embed_xmp_roundtrip_gif`	GIF89a Application Extension XMP roundtrip.
`write_never_creates_sidecar_for_jpeg`	Saving on a JPEG creates zero `.xmp` files.
`write_removes_legacy_sidecar_after_embed`	A pre-existing `.xmp` is removed on successful embed.
`write_preserves_foreign_rating_and_description`	Foreign `xmp:Rating` and `dc:description` survive a tag edit.
`write_errors_on_unsupported_format`	Saving on RAW returns `Err`, no sidecar fallback.

Test	Verifies
`registry_recognises_every_expected_extension`	Every advertised handler is registered.

Run:

```
cd src-tauri
cargo test --test metadata_fs
```

L4 — screenshot smoke tests

Under `scripts/`:

- `screenshot-app.ps1` — launches Magpie, finds its window with `EnumWindows`, screenshots via `PrintWindow` (`PW_RENDERFULLCONTENT`) so the shot works even if another window is in front.
- `screenshot-scrolled.ps1` — clicks an image, scrolls the details panel, screenshots.
- `test-multiselect-tag.ps1` — end-to-end skeleton for Ctrl+click → type tag → click Apply. UI automation is inherently flaky on Windows; use these mostly to verify a build launches and paints, not for asserting behaviour.

These are not part of automated CI. They're one-off tools when investigating a UI-only bug.

Frontend tests

Currently minimal:

- Type safety is the main safety net.
- Component tests would use Vitest + React Testing Library; not set up in v1.

Because the UI is a thin translation over Rust commands, the interesting behaviour is on the Rust side and covered by L2/L3.

Format sample files

`samples/format-samples/` holds one average-sized synthetic file per handler registered in `FormatRegistry` (35 handlers, plus extension aliases such as `.jpeg`, `.tif`, `.m4v`).

Generate or refresh:

```
py -m pip install pillow reportlab pillow-heif imageio-ffmpeg
py samples/format-samples/generate_all.py
```

Output lands in `samples/format-samples/files/<ext>/sample.<ext>` with sizes and placeholder notes recorded in `manifest.json`. See `samples/format-samples/README.md` for caveats (camera RAW and some exotic codecs are scan-only placeholders, not vendor-native files).

Use these when manually exercising the scanner, thumbnail pipeline, or details panel against a broad extension set without copying a real library onto the machine.

Diagnostic tools (not tests, but adjacent)

- `src-tauri/examples/dump_meta.rs` — prints filesystem state, embedded XMP, any legacy sidecar contents, parsed metadata, and DB state for one photo or the first N rows in the DB. Great for reproducing bugs.
- `src-tauri/examples/dump_tag_usage.rs` — given a tag name, list every photo carrying it plus what the on-disk embedded XMP shows. Used in the “did the batch save actually work?” investigation.

Run:

```
cd src-tauri
cargo run --example dump_meta -- "C:\Photos\IMG_1234.jpg"
```

CI (recommended)

Not shipped in the repo yet, but the natural setup is:

- Windows runner (matrix: `windows-2022`).
- Steps: `cargo fmt --check`, `cargo clippy --all-targets`, `cargo test --workspace`, `npm ci`, `tsc -b`, `npm run tauri build -- --no-bundle`.

- Artifacts: `desktop.exe` for smoke inspection.

Build and release

Prerequisites (Windows)

```
# Rust toolchain (user scope)
winget install --id Rustlang.Rustup --accept-source-agreements

# MSVC C++ Build Tools (machine scope; needs UAC)
winget install --id Microsoft.VisualStudio.2022.BuildTools `
  --override "--wait --passive --add Microsoft.VisualStudio.Workload.VCTools --
includeRecommended"

# Node LTS
winget install --id OpenJS.NodeJS.LTS
```

Verify:

```
rustc --version   # ≥ 1.77.2
cargo --version
node --version    # ≥ 18
npm --version
```

Clone and build

```
git clone <repo> magpie
cd magpie
npm install
```

Development

```
npm run tauri dev
```

This starts:

- **Vite dev server** on `localhost:1420` with HMR.
- **Cargo watch** compiling the Rust core.
- **Tauri window** loading the dev server URL.

Rust code changes trigger a full app rebuild (10–30 s); frontend changes hot-reload in <1 s.

Release build

```
npm run tauri build -- --no-bundle
```

- `--no-bundle` skips the MSI/NSIS installer, producing just `src-tauri/target/release/desktop.exe`. Faster and useful for local testing.
- Remove `--no-bundle` (or pass `--bundles msi` / `--bundles nsis`) to produce an installer under `src-tauri/target/release/bundle/`.

Timings on a modern laptop:

- Cold release build: ~5 min.
- Incremental release build after a small Rust change: ~90 s.
- Incremental after frontend change only: ~30 s.

Tauri configuration

Key fields in `src-tauri/tauri.conf.json`:

Field	Notes
`productName`	`Magpie`
`identifier`	`com.magpie.app` — used for `%APPDATA%` folder name.
`security.csp`	Strict; blocks inline scripts and remote origins.
`app.withGlobalTauri`	`false` — commands accessed via `@tauri-apps/api` only.
`bundle.icon`	Multi-size Windows ICO.
`bundle.windows.wix`	(Present in future for signed MSI.)

Capabilities (`src-tauri/capabilities/default.json`) declare only what's needed: dialog and opener plugins, asset access under the user's library folders. Network is *not* granted.

Release profile

`src-tauri/Cargo.toml`:

```
[profile.release]
codegen-units = 16
lto           = false
opt-level     = 3
panic         = "abort"
strip         = true
incremental   = false
```

Rationale:

- `codegen-units = 16` + `lto = false` — trades a bit of runtime perf for much faster incremental builds. In practice, the hot paths are limited by `image` decode and disk I/O, not by cross-crate inlining.
- `panic = "abort"` — smaller binary, no unwinding tables. All panics are treated as bugs; the app crashes and restarts.
- `strip = true` — removes debug symbols to keep the binary small (~13 MB stripped vs. 45 MB with symbols).

Version bump

1. `Cargo.toml`: `version = "0.1.1"` (both files if we ever add a workspace).
2. `package.json`: same.
3. `tauri.conf.json`: `version = "0.1.1"`.
4. Commit as `chore: v0.1.1`.

Releasing

The v1 workflow is manual:

1. `npm run tauri build` (with default bundle) on a clean tree.
2. Test the resulting MSI on a clean VM.
3. Draft a GitHub release with the MSI attached.
4. Attach the two PDFs from `docs/` as documentation assets.
5. Tag the commit `v0.1.0` and publish.

A GitHub Actions workflow is not shipped in v1 but can plug into the recommended CI setup in Testing strategy.